

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**

Высшая школа электроники и компьютерных наук

Кафедра системного программирования

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой, д.ф.-м.н.,
профессор

_____ Л.Б. Соколинский

« ____ » _____ 2023 г.

**Разработка веб-приложения «Тренажер для набора текста
на клавиатуре»**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ЮУрГУ – 09.03.04.2023.308-131.ВКР

Научный руководитель,
доцент кафедры СП, к.т.н.
_____ Н.Ю. Долганина

Автор работы,
студент группы КЭ-404
_____ Е.В. Исайкин

Ученый секретарь
(нормоконтролер)
_____ И.Д. Володченко
« ____ » _____ 2023 г.

Челябинск, 2023 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**

**Высшая школа электроники и компьютерных наук
Кафедра системного программирования**

УТВЕРЖДАЮ

Зав. кафедрой СП

_____ Л.Б. Соколинский

06.02.2023 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

студенту группы КЭ-404

Исайкину Евгению Валерьевичу,

обучающемуся по направлению

09.03.04 «Программная инженерия»

1. Тема работы (утверждена приказом ректора от 25.04.2023 г. № 753-13/12)
Разработка веб-приложения «Тренажер для набора текста на клавиатуре».

2. Срок сдачи студентом законченной работы: 05.06.2023 г.

3. Исходные данные к работе

3.1. Руководство по CSS. [Электронный ресурс] URL: <https://developer.mozilla.org/ru/docs/Web/CSS/Reference> (дата обращения: 03.02.2023 г.).

3.2. Руководство JavaScript. [Электронный ресурс] URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 06.02.2023 г.).

3.3. JavaScript. Полное руководство, 7-е изд.: Пер. с англ. – СПб.: ООО «Диалектика», 2021. – 720с.

3.4. React: современные шаблоны для разработки приложений. 2-е изд. – СПб.: Питер, 2022. – 320с.

3.5. Документация React. [Электронный ресурс] URL: <https://ru.reactjs.org/docs/getting-started.html> (дата обращения: 03.02.2023 г.).

3.6. JavaScript и Node.js для веб-разработчиков / Н. А. Прохоренко, В. А. Дорнов. – СПб.: БВ-Петербург, 2022 г. – 767с.

4. Перечень подлежащих разработке вопросов

4.1. Анализ предметной области.

4.2. Разработка требований к системе.

4.3. Проектирование веб-приложения.

- 4.4. Реализация веб-приложения.
- 4.5. Тестирование веб-приложения.
- 5. Дата выдачи задания: 06.02.2023 г.

Научный руководитель,
доцент кафедры СП, к.т.н.

Н.Ю. Долганина

Задание принял к исполнению

Е.В. Исайкин

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	7
1.1. Обзор аналогичных проектов.....	7
1.2. Инструменты и технологии	18
2. ТРЕБОВАНИЯ К СИСТЕМЕ	20
2.1. Функциональные требования	20
2.2. Нефункциональные требования.....	21
3. ПРОЕКТИРОВАНИЕ	22
3.1. Диаграмма вариантов использования.....	22
3.2. Проектирование макетов	24
3.3. Диаграмма развертывания	25
3.4. Проектирование базы данных	25
4. РЕАЛИЗАЦИЯ.....	28
4.1. Верстка макетов сайта.....	28
4.2. Реализация тренировки.....	31
4.3. Реализация изменения темы	33
4.4. Управление состоянием	35
4.5. Реализация REST API.....	37
4.6. Подключение к базе данных.....	40
5. ТЕСТИРОВАНИЕ	42
5.1. Функциональное тестирование	42
5.2. Тестирование кроссбраузерности	44
ЗАКЛЮЧЕНИЕ.....	45
ЛИТЕРАТУРА	46
ПРИЛОЖЕНИЯ	48
Приложение А. Скриншоты веб-приложения	48
Приложение Б. Функция для тренировки печати.....	56

ВВЕДЕНИЕ

Актуальность

Развитие общества на сегодняшний день немыслимо без использования персональных компьютеров. Каждый день сотни миллионов пользователей вводят информацию или же управляют работой различных технических устройств с помощью компьютерной периферии.

Клавиатура – это одна из самых важных частей компьютерной периферии. При работе за персональным компьютером пользователь использует клавиатуру как устройство для ввода команд, печати текста и т.д.

Скорость печати значительно увеличивает продуктивность, поэтому для любого пользователя высокая скорость печати – это необходимый навык в современном мире.

Для достижения высокой скорости необходимо тренироваться и использовать популярный метод печати – слепой десятипальцевый метод. Смысл данного метода заключается в том, что при наборе текста пользователь, не глядя на клавиши задействует все пальцы рук или большую их часть.

Реализация веб-приложения «Тренажер для набора текста на клавиатуре» не только позволит повысить скорость печати текста, ввиду постоянных тренировок, но и также научиться слепому десятипальцевому методу набора текста.

Постановка задачи

Целью выпускной квалификационной работы является разработка веб-приложения «Тренажер для набора текста на клавиатуре». Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ предметной области;
- 2) разработать требования к веб-приложению;
- 3) выполнить проектирование веб-приложения;
- 4) реализовать веб-приложение;
- 5) провести тестирование веб-приложения.

Структура и содержание работы

Работа состоит из введения, пяти глав, заключения и списка литературы. Объем работы составляет 59 страниц, объем списка литературы – 19 источников.

В первой главе описывается анализ предметной области – обзор аналогичных проектов, а также используемые инструменты для разработки.

Вторая глава посвящена функциональным и нефункциональным требованиям.

В третьей главе рассмотрен процесс проектирования системы. Приведена диаграмма вариантов использования и диаграмма развертывания. А также спроектированы макеты страниц веб-приложения и база данных.

В четвертой главе приведена реализация веб-приложения.

В пятой главе представлены результаты тестирования разработанного веб-приложения.

В заключении представлены результаты проделанной работы.

В приложении А содержатся скриншоты веб-приложения.

В приложении Б содержится функция для тренировки печати.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Обзор аналогичных проектов

Для создания веб-приложения, которое сможет привлечь достаточную базу пользователей и станет удобным инструментом для тренировок скорости печати, необходимо проанализировать уже существующие аналогичные проекты, которые успешно функционируют. Это необходимо для определения базовых требований и выявления достоинств и недостатков уже существующих проектов.

В ходе выполнения поставленной задачи были найдены несколько наиболее популярных приложений, предоставляющих схожий по задумке функционал. В качестве наиболее подходящих были выделены следующие проекты: «STAMINA-ONLINE», «Соло на клавиатуре», «Touch Typing Study».

Рассмотрим подробнее следующие проекты для набора текста на клавиатуре.

«STAMINA-ONLINE» [1]

«STAMINA-ONLINE» – это бесплатный веб-тренажер, целью которого является предоставление обучающих материалов и различных тренировок по набору текста на клавиатуре. Плюсом данного тренажера является неограниченная свобода действий, а также поэтапное обучение. Пользователь сам выбирает с чего начать и какой режим тренировки выбрать. Сервис имеет поддержку русской и английской раскладок. Основную часть сайта занимает информация о функциональных возможностях и особенности обучения слепому десятипальцевому методу (рисунок 1).

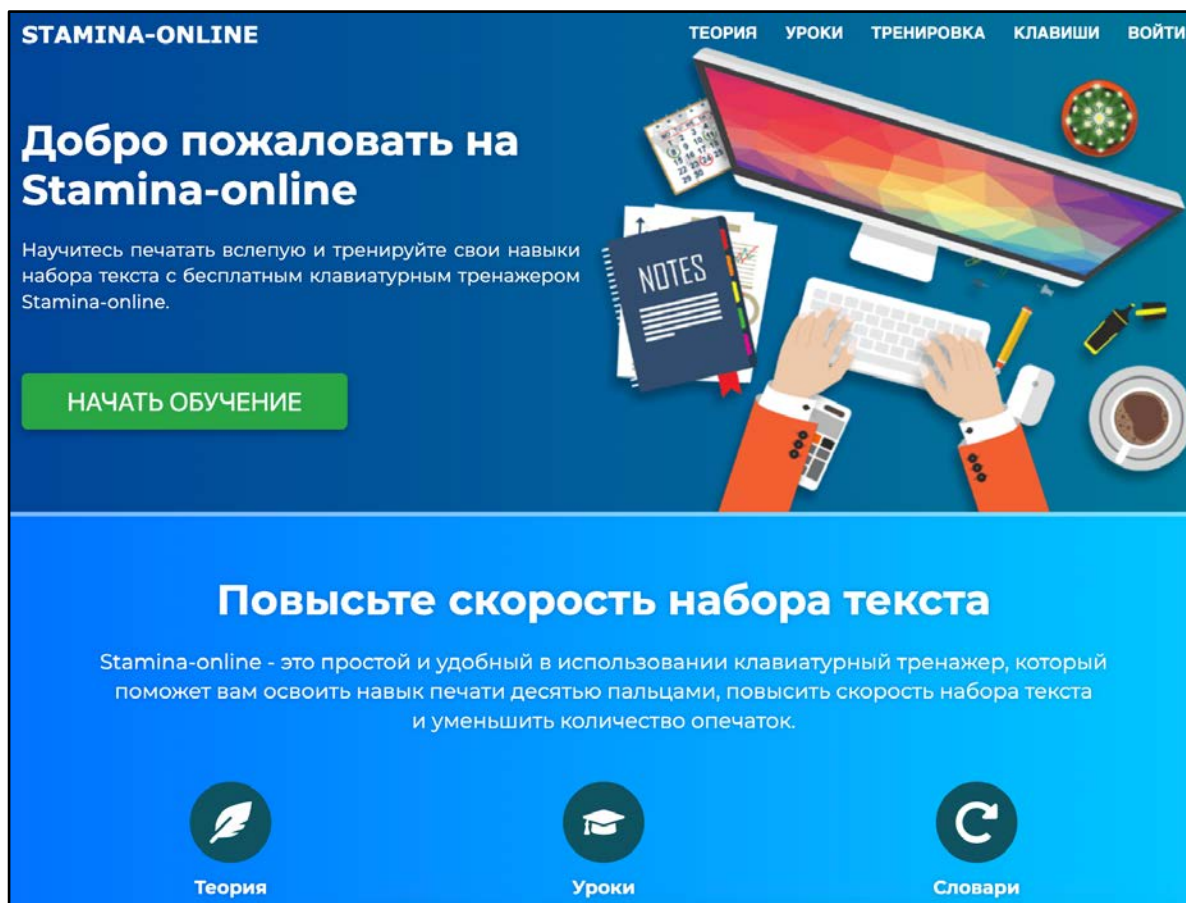


Рисунок 1 – Главная страница сайта «STAMINA-ONLINE»

Меню сайта разделено на пункты «Теория», «Уроки», «Тренировка», «Клавиши» и «Войти». Также имеется функциональная кнопка, закрепленная на логотипе, которая ведет на главную страницу сайта. Сайт адаптивно понятен для использования, не возникает вопросов как найти интересующий элемент. При нажатии на кнопку «Уроки» нас переносит на вкладку с тренировками различных аспектов клавиатуры (рисунок 2). Есть возможность создания аккаунта для сохранения своего прогресса по пройденным урокам.

На вкладке «Теория» можно ознакомиться с различной информацией касающейся тематики слепой печати.

При нажатии на кнопку «Тренировка» открывается страница с тренировкой печати. На данной странице расположена строка с текстом, динамическая клавиатура, которая указывает на какие клавиши нужно нажимать и

две руки (рисунок 3). Каждый цвет пальцев рук должен покрывать клавиши с таким же цветом. Минусом является то, что нельзя включить пропуск ошибок, а необходимо их исправлять, чтобы дальше печатать текст.

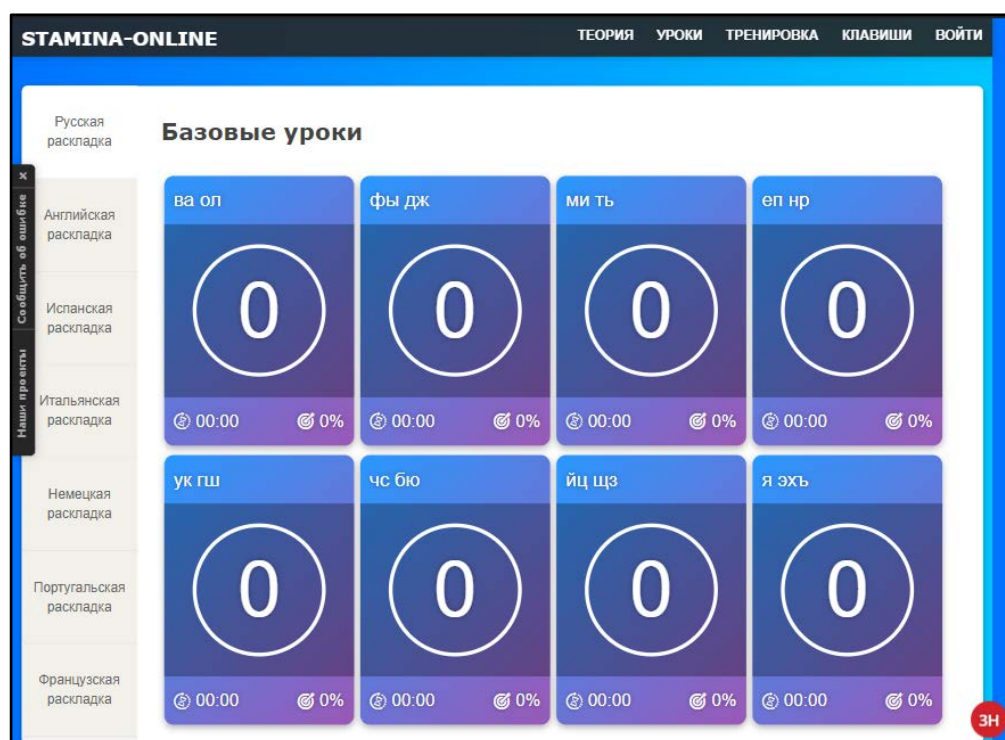


Рисунок 2 – Страница с вариациями различных уроков

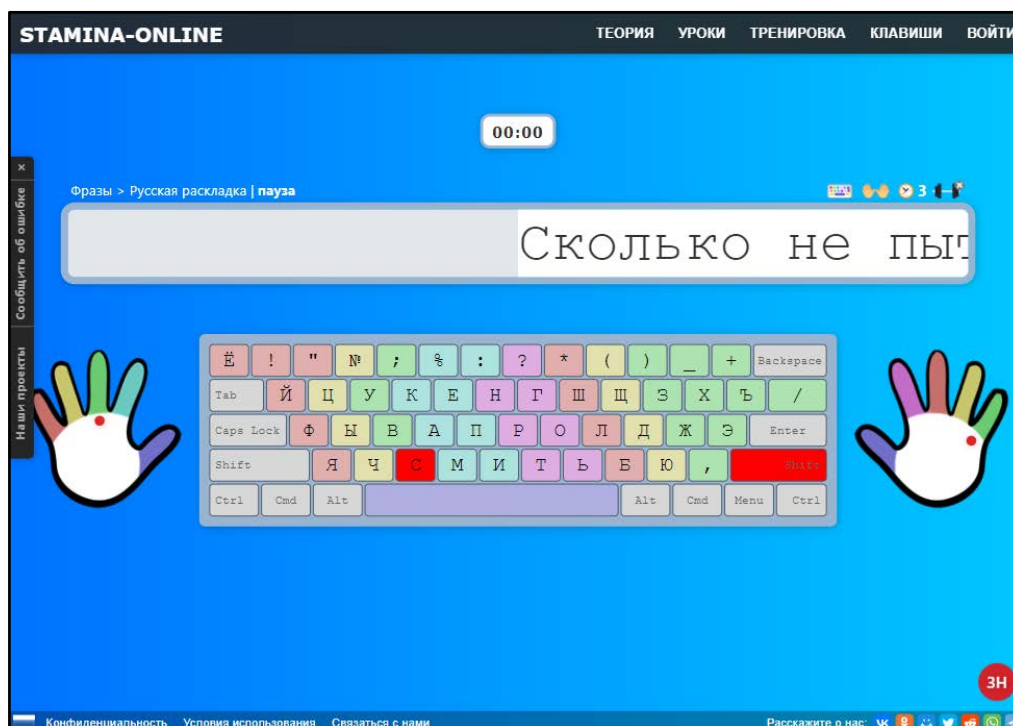


Рисунок 3 – Страница тренировки печати

После выполнения тренировки вам будет предоставлена информативная статистика, которая покажет скорость вашей печати, время, затраченное на тренировку, количество символов и процент точности (рисунок 4).

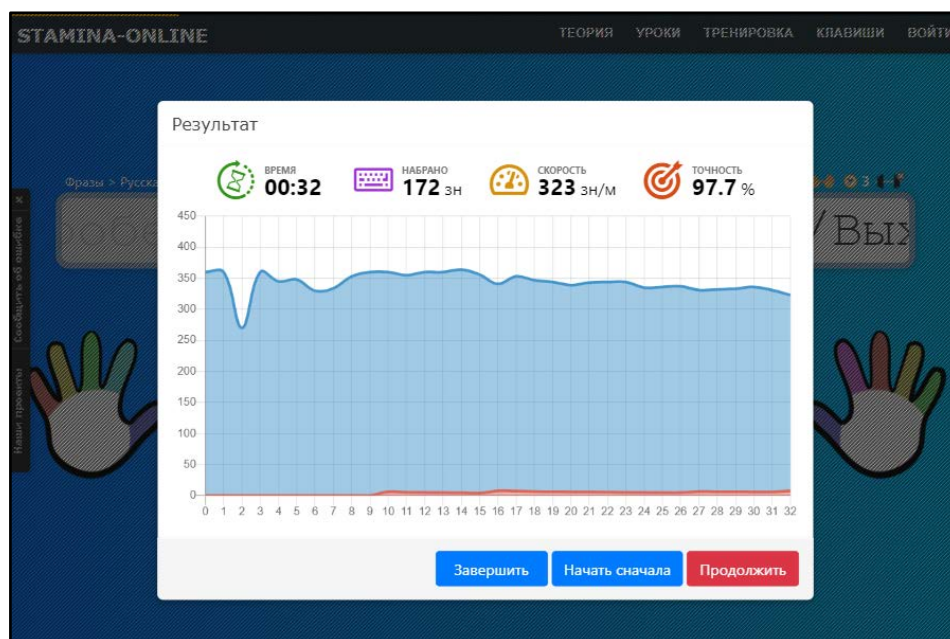


Рисунок 4 – Статистика тренировки на «STAMINA-ONLINE»

Для еще большей продуктивности в работе на вкладке «Клавиши» можно рассмотреть список клавиш быстрого доступа для операционных систем, а также для различных программ (рисунок 5).

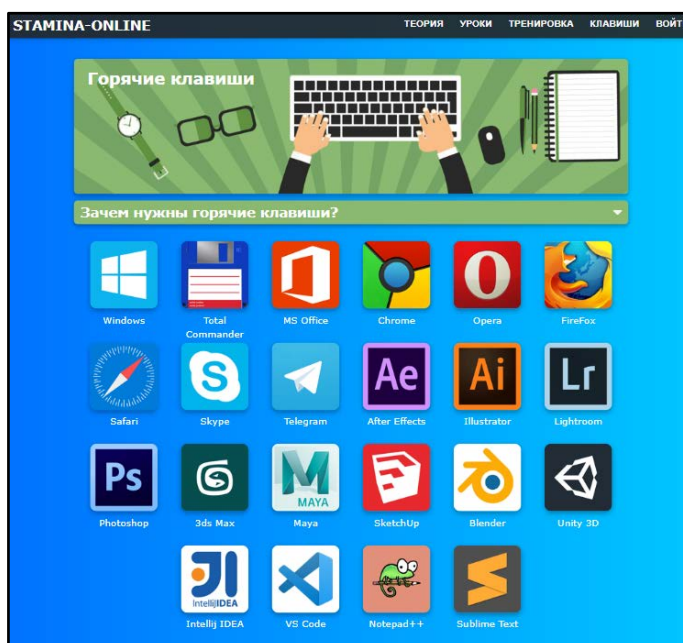


Рисунок 5 – Вкладка «Клавиши»

Сайт не имеет адаптивного дизайна (рисунок 6), но он не нужен, так как мы практикуем печать на клавиатуре, что априори подразумевает наличие настольного компьютера.



Рисунок 6 – Мобильная версия «STAMINA-ONLINE»

«Соло на клавиатуре» [2]

«Соло на клавиатуре» – это, пожалуй, один самых известных клавиатурных тренажеров. Но данный проект является коммерческим, поэтому вся программа платная. Обучение, помимо русского и английского языков, доступно и на немецком, британском, итальянском, испанском. Главную страницу занимает представление продаваемого курса (рисунок 7).

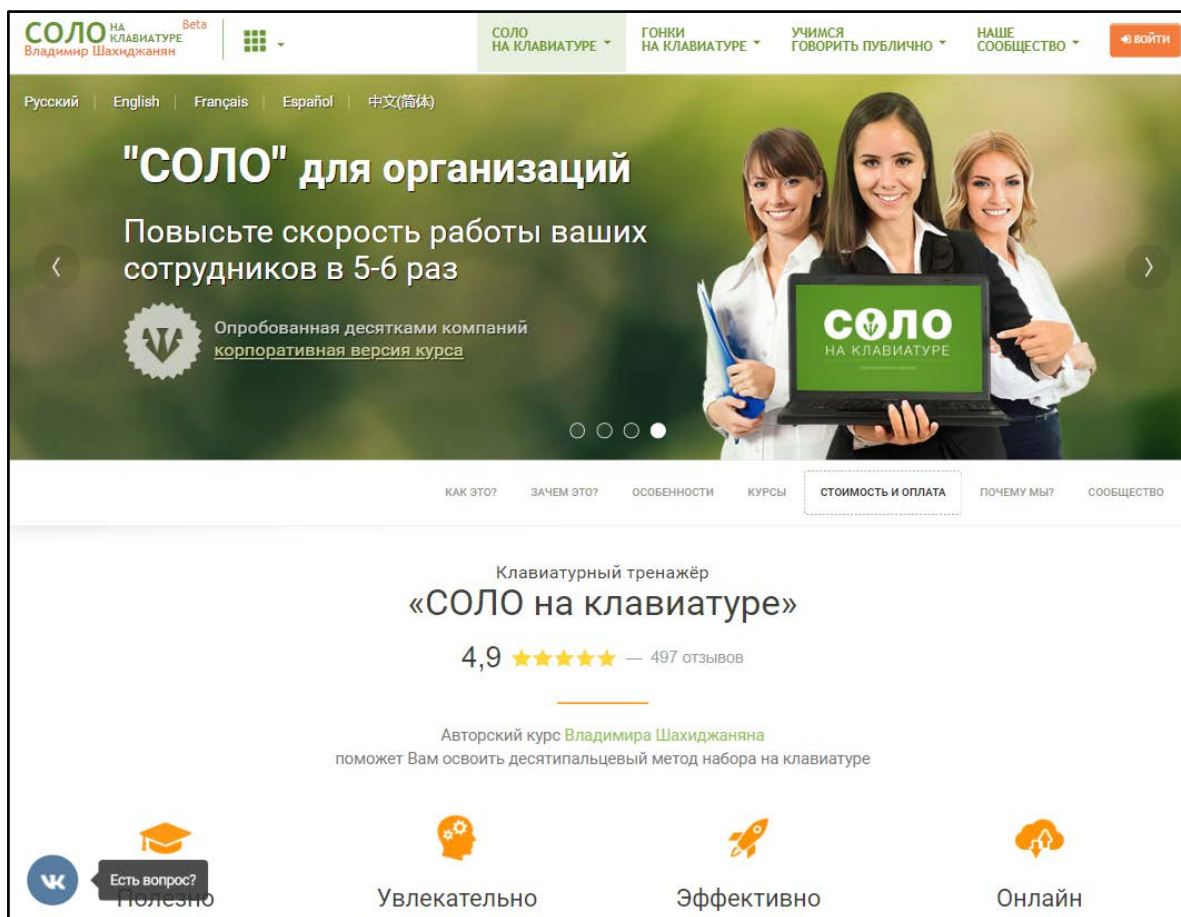


Рисунок 7 – Главная страница «Соло на клавиатуре»

Зарегистрированным пользователям доступно только несколько первых уроков. Но перед тем, как приступить к тренировкам пользователю дается возможность пройти тест на скорость набора текста, который позволит оценить навык печати и узнать свои слабые места. Также из функциональных возможностей имеется аудиодиктант, который позволит пользователю попробовать свои силы в печати текста под диктовку.

В сам тренажер входит поле с тренировочным текстом, клавиатура, с залитыми разными цветами клавишами, которые выступают в роли указателей для пальцев, статистика, которая обновляется в реальном времени и показывает скорость печати, процент ошибок и время (рисунок 8). Есть возможность настройки тренажера под себя, но нет возможности пропуска ошибок.

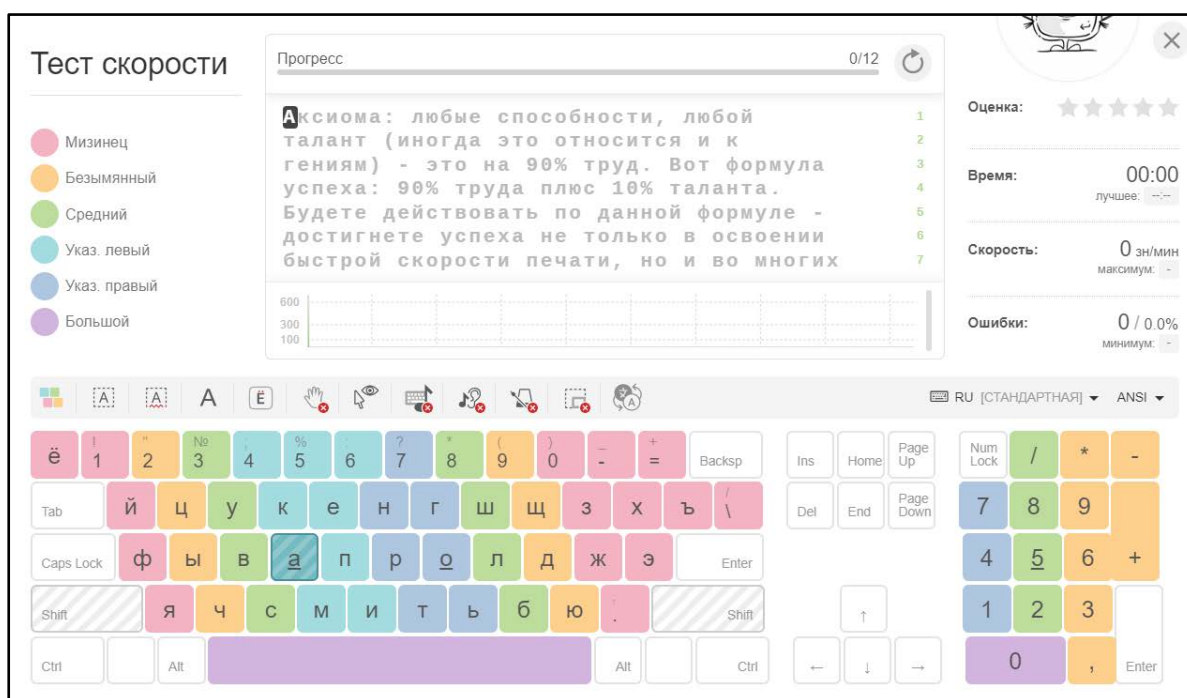


Рисунок 8 – Тренажер печати «Соло на клавиатуре»

«Touch Typing Study» [3]

Touch Typing Study» – это еще один бесплатный веб-тренажер печати с понятным интерфейсом. На главной странице тренажера нас встречает кнопка «Начните печатать» и широкий выбор раскладок визуализированной клавиатуры (рисунок 9). В правой части имеется меню, в котором находятся уроки по обучению, тесты и различная информация, связанная с тематикой слепой печати. Также можно выбрать любой язык интерфейса из обширного списка поддерживаемых языков тренажера. А для сохранения и отслеживания своего прогресса на сайте имеется возможность создания своей учетной записи.

Обучение навыку печати состоит из восемнадцати уроков, которые можно проходить в любом порядке. Каждый урок содержит в себе несколько упражнений на отработку материала. Во время обучения имеется возможность проверить свою скорость печати на словах, которые никак не связаны между собой, а также тест печати текста.

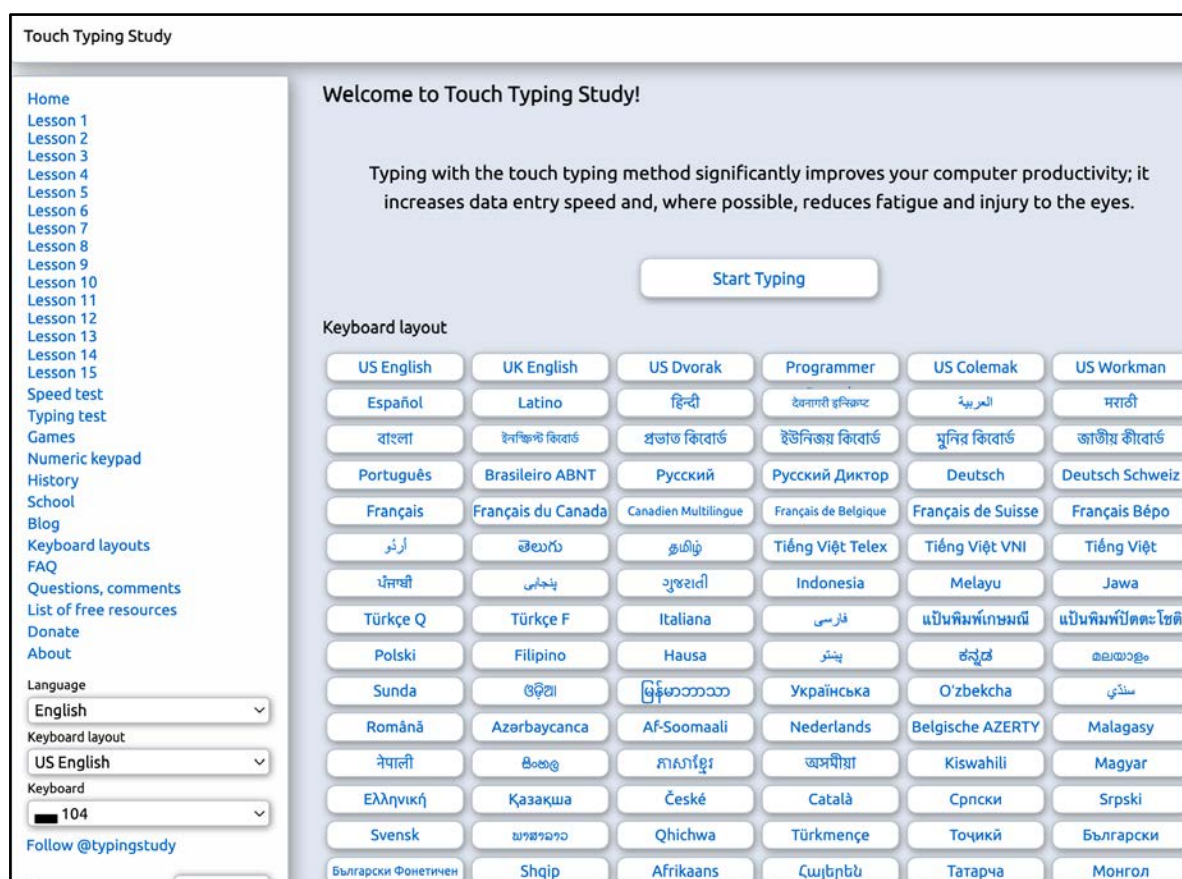


Рисунок 9 – Главная страница «Touch Typing Study»

Интерфейс тренажера довольно простой. В нем находится поле с текстом или упражнением, пустое поле в котором мы набираем текст и виртуальная клавиатура, которая подсказывает на какие клавиши необходимо нажимать в данный момент (рисунок 10). Во время прохождения обучения нам доступна динамическая информация с нашей статистикой. А после упражнения дается подробная информация по каждому символу из данного урока (рисунок 11).

Из плюсов хотелось бы выделить то, что при работе с тестами на скорость печати или же при наборе текста имеется возможность не исправлять ошибки принудительно, а набирать текст дальше тем самым экономя время. Количество ошибок считается посимвольно, а слова, написанные с ошибками, окрашиваются в красный цвет.

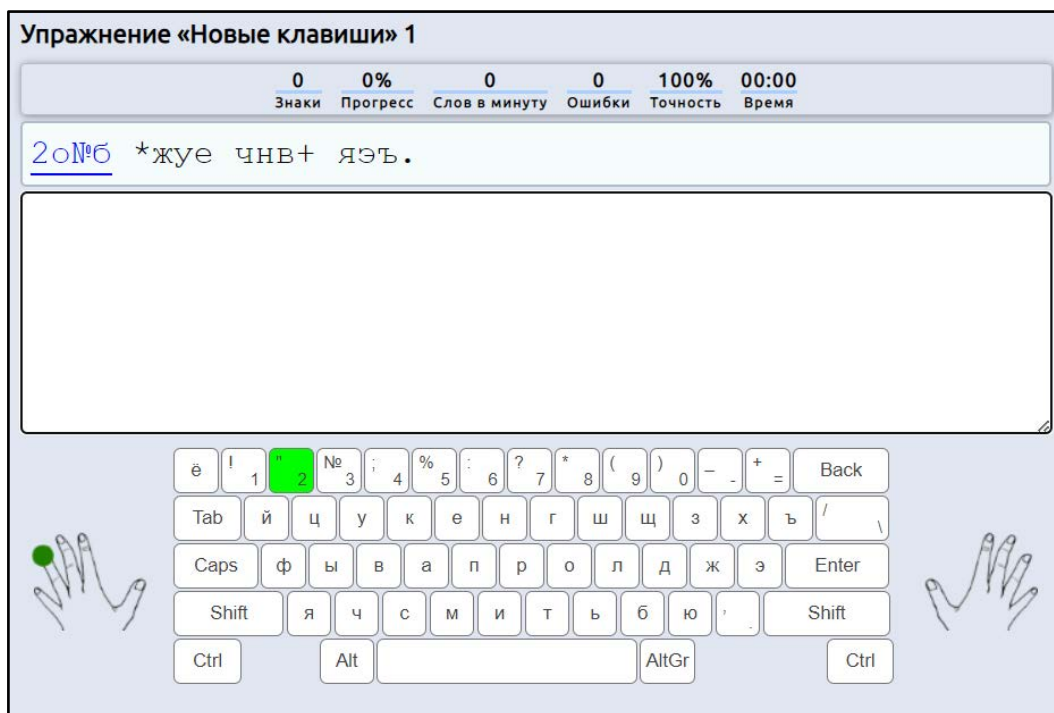


Рисунок 10 – Тренажер печати «Touch Typing Study»

Упражнение на время 01:12		
Ваша текущая скорость печати 45 слов в минуту		
г	0/1	0%
,	2/3	66%
д	5/6	83%
к	7/8	87%
а	18/19	94%
о	26/27	96%
в	1/1	100%
я	6/6	100%
э	5/5	100%
ы	5/5	100%
е	17/17	100%
с	1/1	100%
"	2/2	100%
=	4/4	100%
б	2/2	100%
н	16/16	100%
ч	5/5	100%
т	17/17	100%
п	11/11	100%
р	9/9	100%

Рисунок 11 – Посимвольная статистика упражнения

Помимо тренажеров по набору текста у приложения «Touch Typing Study» имеется тренажер по работе с цифровым блоком (рисунок 12).

Тренажер по работе с цифровым блоком предлагает упражнения, в которых пользователь будет набирать числа и выполнять различные операции, используя цифровую клавиатуру. Это может быть полезным для тех, кто часто работает с числовыми данными, выполняет расчеты или работает в финансовой сфере.

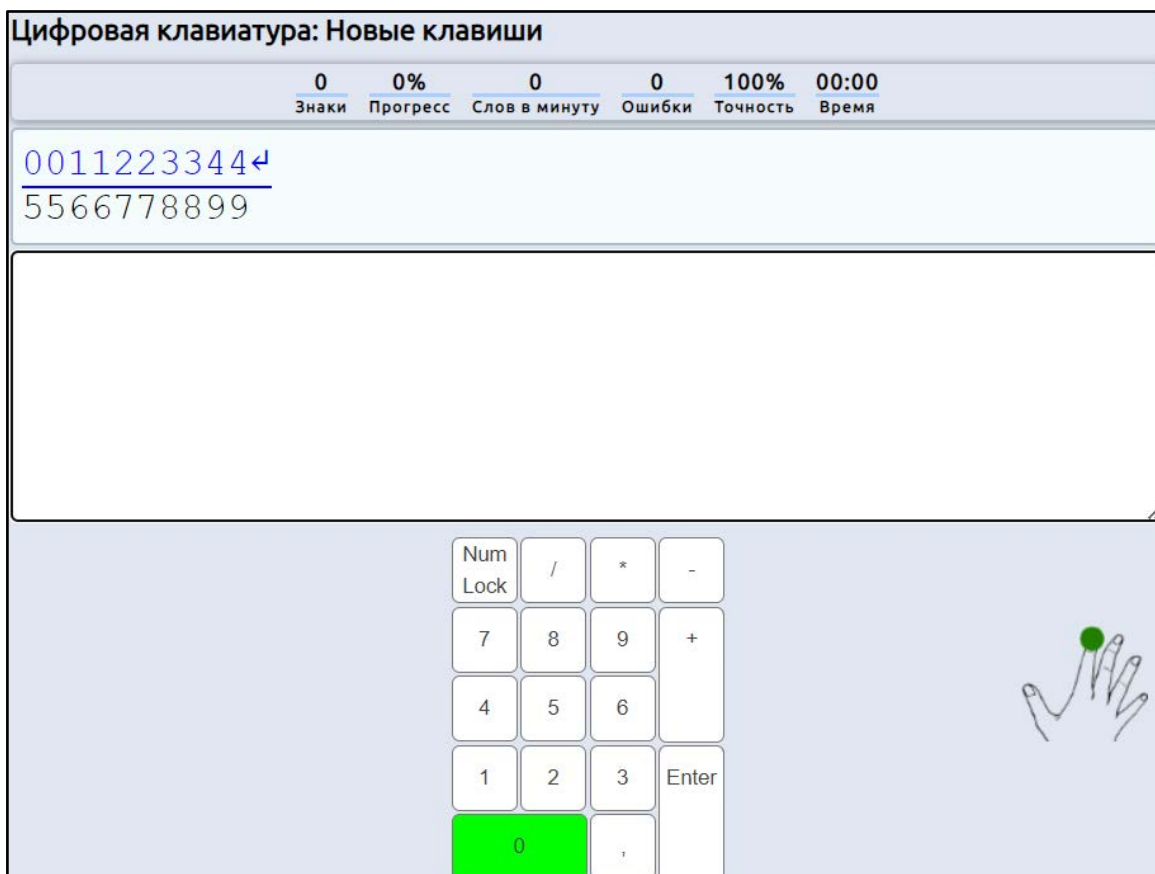


Рисунок 12 – Тренажер цифрового блока «Touch Typing Study»

Как и у всех тренажеров, которые мы рассмотрели в данной главе, у тренажера «Touch Typing Study» отсутствуют версии, специально адаптированные под разрешения, отличные от настольного компьютера. Это означает, что тренажер может быть оптимально использован только на компьютере, а не на мобильных устройствах или планшетах (рисунок 13).

1.2. Инструменты и технологии

Веб-приложение «Тренажер для набора текста на клавиатуре» – это приложение, которое будет спроектировано и реализовано по уникальному шаблону. Необходимо реализовать две составляющие: клиентскую часть – это то, с чем взаимодействует пользователь напрямую в браузере, и серверную – это то, что будет обрабатывать информацию, полученную от пользователя на клиентской части.

В качестве редактора кода выбран Visual Studio Code [4], так как он имеет мощные возможности, такие как подсветка синтаксиса, автозаполнение кода, применение различных расширений для ускорения разработки и улучшения качества кода, интеграция с системой контроля версий Git [5].

Реализация клиентской части приложения будет разработана с использованием следующих технологий.

HTML (от англ. HyperText Markup Language – «язык гипертекстовой разметки») – стандартизированный язык разметки документов, с помощью которого формируется структура страницы веб-ресурса. Язык HTML интерпретируется браузерами, полученный в результате интерпретации форматированный текст отображается на экране компьютера или мобильного устройства [6].

CSS (от англ. Cascading Style Sheets – «каскадные таблицы стилей») – язык стилей, который описывает внешний вид документа, написанного с использованием языка разметки HTML [7].

SCSS (от англ. Sassy Cascading Style Sheets – «дерзкие каскадные таблицы стилей») – это препроцессор, который расширяет функциональность CSS и предоставляет удобные инструменты, такие как переменные, миксины и вложенность правил. Это помогает упростить и организовать структуру стилей [8].

JavaScript – основной язык программирования Всемирной паутины, который позволяет придавать веб-страницам дополнительную интерактивность. А также поддерживает объектно-ориентированные, императивные и функциональные стили [9].

React – это JavaScript-библиотека для создания пользовательских интерфейсов. С ее помощью можно разделить интерфейс на множество компонентов и эффективно управлять их состоянием и обновлением. React также обладает встроенной виртуальной DOM и механизмом перерисовки, что повышает производительность приложения [10].

Redux – это библиотека для управления состоянием приложения в JavaScript. Она предоставляет единое хранилище состояния, которое позволяет централизованно управлять данными приложения и обновлять интерфейс на основе этих данных [11].

Реализация серверной части приложения будет разработана с использованием следующих технологий.

Node.js – это среда выполнения JavaScript на сервере, которая позволяет разрабатывать серверные приложения [12].

Express – фреймворк для разработки серверных приложений на Node.js. Он облегчает создание маршрутов, обработку запросов и управление ресурсами на сервере [13].

MongoDB – документноориентированная система управления базами данных, которая используется для хранения данных приложения [14].

Вывод по первой главе

В первой главе был проведен анализ аналогичных проектов. Каждый из этих проектов имеет свои недостатки, их необходимо учесть при проектировании и разработке. Также был произведен выбор инструментов разработки веб-приложения.

2. ТРЕБОВАНИЯ К СИСТЕМЕ

2.1. Функциональные требования

К функциональным требованиям относятся сервисы, которые выполняет система. Должно быть указано, как именно реагирует система на те или иные выходные данные, как она ведет себя в определенных ситуациях. Иногда указывается, что система не должна делать.

Функциональные требования.

1. Система должна предоставлять пользователю возможность переключать тему приложения.
2. Система должна предоставлять пользователю возможность выбора режима тренировки.
3. Система должна предоставлять пользователю возможность включение подсветки текста.
4. Система должна предоставлять пользователю возможность выбрать язык текста (русский или английский).
5. Система должна предоставлять пользователю возможность включение подсветки клавиатуры.
6. Система должна предоставлять пользователю его статистику после завершения тренировки, включающая общее количество набранных символов, символов в минуту, время тренировки, точность ввода, язык текста и выбранный режим.
7. Система должна предоставлять авторизованному пользователю автоматическую запись статистики тренировки в базу данных.
8. Система должна предоставлять авторизованному пользователю просмотр всех своих тренировок.
9. Система должна предоставлять пользователю просмотр рейтинга тренировок.
10. Система должна предоставлять авторизованному пользователю просмотр и редактирование своей личной информации.

11. Система должна предоставлять авторизованному пользователю удалять свои тренировки.
12. Система должна предоставлять неавторизованному пользователю возможность зарегистрироваться на сайте.
13. Система должна предоставлять неавторизованному пользователю возможность авторизоваться на сайте.

2.2. Нефункциональные требования

К нефункциональным требованиям относят характеристики системы и ее окружения, не относящиеся к поведению системы. Здесь указывается перечень ограничений, накладываемых на действия и функции, выполняемые системой.

Нефункциональные требования.

1. Приложение должно корректно открываться в таких браузерах: Яндекс.Браузер, Google Chrome, Mozilla Firefox, Microsoft Edge, Safari.
2. Интерфейс должен быть интуитивно понятным и легким в использовании для пользователя.
3. Дизайн приложения должен быть привлекательным.
4. Веб-приложение должно быть разработано с использованием таких инструментов, как: HTML, CSS, SCSS, JavaScript, React, Redux, Node.js, Express, MongoDB.

Вывод по второй главе

Во второй главе с учетом обзора аналогичных проектов и поставленных задач были определены и сформулированы функциональные и нефункциональные требования для веб-приложения.

3. ПРОЕКТИРОВАНИЕ

3.1. Диаграмма вариантов использования

Для проектирования поведения пользователей разрабатываемого веб-приложения с учетом сформулированных ранее функциональных требований была составлена диаграмма вариантов использования (рисунок 14). Диаграмма вариантов использования (от англ. use-case) – это графическое представление, необходимое для отображения взаимоотношений зависимостей между группами вариантов использования и действующих лиц, участвующих в процессе [15].

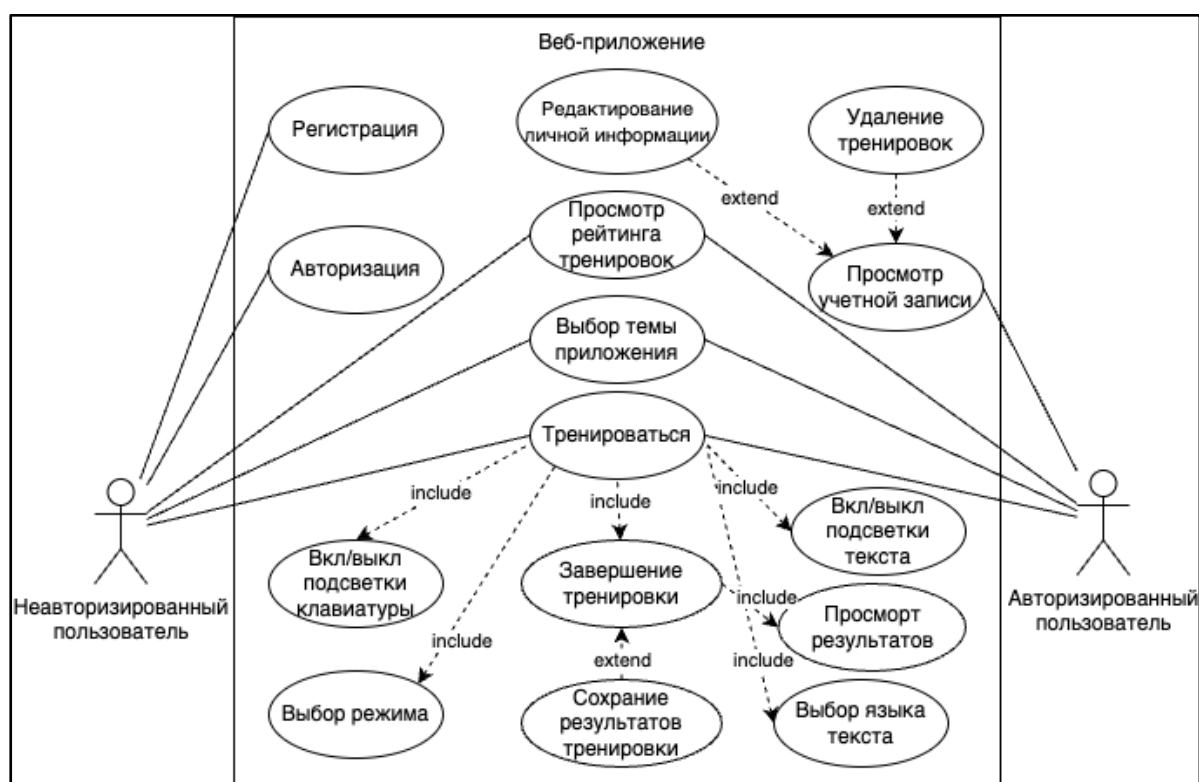


Рисунок 14 – Диаграмма вариантов использования

В проектируемой системе представлены два актера.

1. Неавторизованный пользователь – это любой пользователь, не прошедший авторизацию или регистрацию в приложении.
2. Авторизованный пользователь – это любой пользователь, который авторизован в системе.

Краткое описание вариантов использования.

1. Регистрация. Неавторизированный пользователь может зарегистрироваться, предоставив необходимые данные.
2. Авторизация. Неавторизированный пользователь может войти в систему, используя свои учетные данные.
3. Просмотр рейтинга тренировок. Пользователь может просматривать рейтинг тренировок, чтобы увидеть лучшие результаты всех пользователей.
4. Выбор темы приложения. Пользователь может выбрать темную или светлую тему.
5. Тренировка печати. Пользователь может тренироваться в слепой печати.
6. Вкл/выкл подсветки клавиатуры. Во время тренировки пользователь может включать или же выключать заливку клавиатуры различными цветами, которые позволяют смотреть какой палец должен отвечать за какие клавиши.
7. Вкл/выкл подсветки текста. Во время тренировки пользователь может включать или же выключать подсветку текста, чтобы видеть уже напечатанные символы и текущий символ для нажатия.
8. Выбор режима. Во время тренировки пользователь может выбрать обычный или строгий режим.
9. Выбор языка текста. Во время тренировки пользователь может выбрать язык текста для печати.
10. Завершение тренировки. Пользователь может завершить тренировку нажатием на клавишу «Enter» или же введя весь текст.
11. Просмотр результатов. Как только пользователь завершил тренировку, он может просмотреть свою статистику по тренировке.
12. Сохранение результатов тренировки. Если пользователь авторизован, то автоматически происходит сохранение результатов тренировки.

13. Просмотр учетной записи. Авторизированный пользователь может просмотреть свою учетную запись, включая личную информацию и сохраненные тренировки.

14. Удаление тренировок. Авторизированный пользователь в личном кабинете может удалять свои тренировки.

15. Редактирование личной информации. Авторизированный пользователь может редактировать личную информацию.

3.2. Проектирование макетов

Перед началом верстки веб-приложения необходимо разработать макеты всех страниц, чтобы наглядно посмотреть дизайнерские решения и облегчить верстку. Их схема отображается в виде основных блочных элементов страницы.

Для всех страниц сайта были спроектированы макеты с помощью онлайн сервиса для разработки интерфейсов Figma [16]. Ссылка на макеты сайта: <https://inlnk.ru/zag1N5> [17].

Все страницы содержат одинаковый по стилю элемент – шапку приложения. В ней расположен логотип и навигационное меню. Контент, который располагается ниже шапки, изменяется в зависимости от функционала страницы.

На главной странице расположена кнопка, которая перенаправляет пользователя на страницу с тренировкой набора текста на клавиатуре. На странице рейтинга представлена таблица лучших тренировок с именем пользователя и его статистикой. На странице тренажера представлен набираемый текст, настройки тренажера и визуализация клавиатуры. На странице авторизации и регистрации представлена форма ввода информации. На странице учетной записи представлена информация о пользователе и таблица о всех его тренировках.

3.3. Диаграмма развертывания

Классические веб-приложения используют трехзвенную архитектуру: в браузере пользователя отображается клиентский интерфейс, включающий в себя страницы сайта. Затем сервер ожидает сообщения с запросами от клиентов, обрабатывая данные, хранящиеся в базе данных. После чего отвечает веб-браузеру через сообщения с HTTP-ответом. Сама же база данных хранится на отдельном сервере. Исходя из этого была спроектирована диаграмма развертывания (рисунок 15).

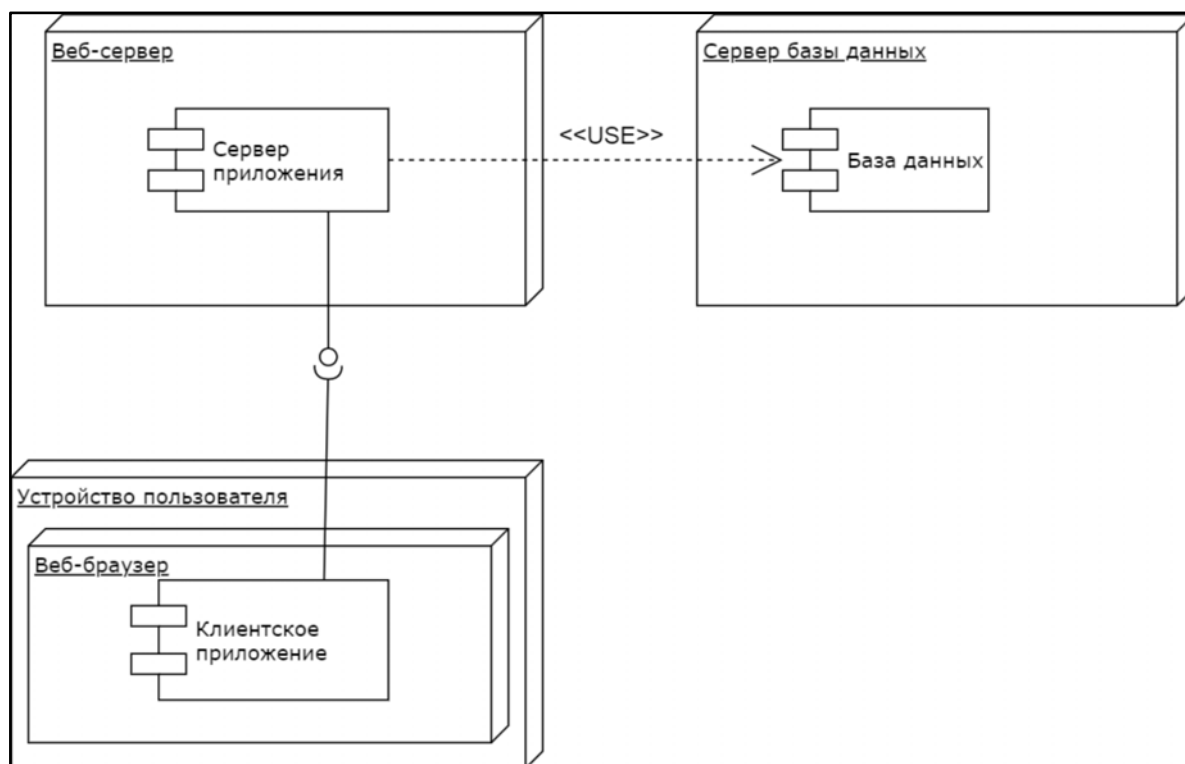


Рисунок 15 – Диаграмма развертывания

3.4. Проектирование базы данных

База данных (БД) – это основная составляющая структуры веб-приложения, предназначенная для хранения, изменения и обработки взаимосвязанной информации, которая необходима для функционирования веб-приложения.

В качестве хранилища данных выбрана СУБД MongoDB, так как она является базой данных NoSQL, что означает, что она не требует строгой

предварительной настройки. Это позволяет гибко изменять структуру данных в процессе разработки, что упрощает и ускоряет работу.

База данных должна хранить информацию обо всех пользователях и их тренировках.

Каждый пользователь должен иметь уникальный идентификатор, имя и почту, а также пароль. У каждого пользователя имеется дата создания и дата обновления.

Каждая тренировка должна включать в себя уникальный идентификатор тренировки, общее количество набранных символов, время тренировки режим и язык вводимого текста. Также каждая тренировка хранит скорость, которая рассчитывается как количество набранных символов, деленное на время тренировки и точность, которая рассчитывается как количество правильно введенных символов, деленное на общее количество символов. А еще имеется дата создания каждой тренировки.

Исходя из вышеперечисленных требований, была спроектирована база данных, содержащая 2 таблицы (рисунок 16).

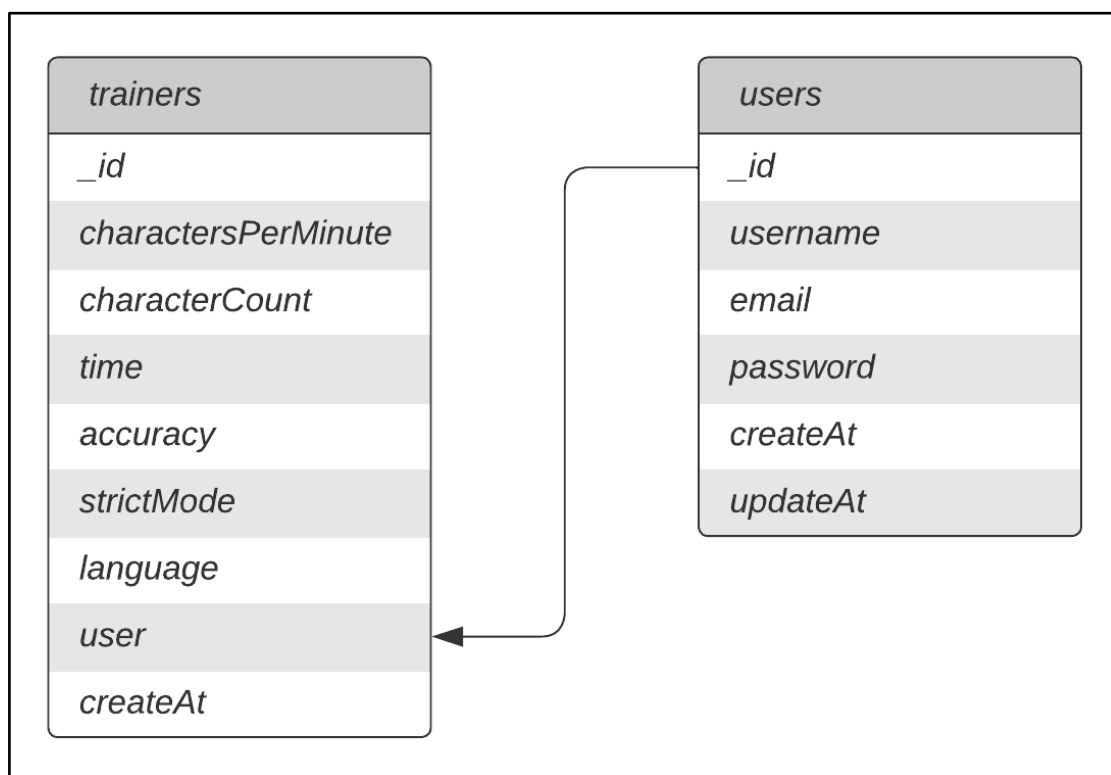


Рисунок 16 – Схема базы данных

В таблице «users» содержатся:

- 1) `_id`: тип `objectId`, идентификатор пользователя;
- 2) `username`: тип `string`, уникальное имя пользователя;
- 3) `email`: тип `string`, уникальная почта пользователя;
- 4) `password`: тип `string`, хешированный пароль пользователя;
- 5) `createAt`: тип `date`, время создания пользователя;
- 6) `updateAt`: тип `date`, время обновления данных пользователя.

В таблице «trainers» содержатся:

- 1) `_id`: тип `objectId`, идентификатор тренировки;
- 2) `charactersPerMinute`: тип `int`, символов в минуту;
- 3) `characterCount`: тип `int`, общее количество символов;
- 4) `time`: тип `int`, время тренировки;
- 5) `accuracy`, тип `int`, точность тренировки;
- 6) `strictMode`, тип `bool`, включенность строгого режима тренировки;
- 7) `language`, тип `string`, язык текста тренировки;
- 8) `user`: тип `objectId`, идентификатор пользователя;
- 9) `createAt`: тип `date`, время создания тренировки.

Вывод по третьей главе

В ходе проектирования веб-приложения были составлены диаграммы вариантов использования и деятельности, а также спроектированы макеты веб-приложения и разработана схема базы данных.

4. РЕАЛИЗАЦИЯ

4.1. Верстка макетов сайта

Верстка – это структурированное сочетание всех элементов в приложении. Верстка в React с использованием SCSS является процессом оформления компонентов приложения. React предоставляет возможность создавать компоненты, которые представляют собой независимые и переиспользуемые строительные блоки пользовательского интерфейса.

В React компоненты создаются с использованием языка разметки JSX, который позволяет объединять HTML-подобный код с JavaScript-логикой. JSX позволяет описывать структуру компонента, определять его свойства и взаимодействие с пользователем.

При верстке компонентов в React, элементы JSX и SCSS структурируются и оформляются таким образом, чтобы достичь желаемого внешнего вида и взаимодействия с пользователем. Каждый компонент может иметь свои собственные стили, которые определяются в отдельных SCSS-файлах и применяются к соответствующим элементам JSX.

Ниже представлен результат использования четырех компонентов статистики после стилизации с помощью SCSS-свойств (рисунок 17). Исходный код компонента представлен в листинге 1.

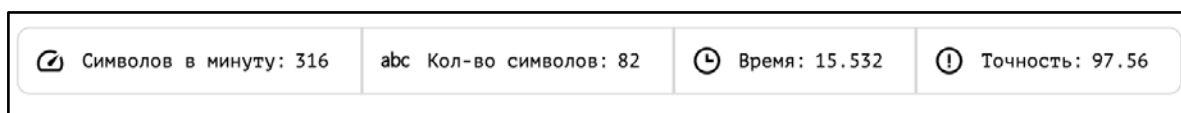


Рисунок 17 – Компоненты статистики

Листинг 1 – Компонент статистики

```
import React from 'react';
import styles from './Statistics.module.scss';
export const Statistics = ({
  icon: Icon = null,
  name = '',
  value = '',
  first = false,
  last = false,
  shortsStat = false,
  onclick = null,
}) => {
```

```

const indicatorsClassName = `${styles.indicators} ${
  shortsStat ? styles.shortsStat : ''
} ${first ? styles.first : ''} ${last ? styles.last : ''}
${onclick ? styles.onclick : ''}`;
const indicatorClassName = `${styles.indicator} ${
  first ? styles.first : ''
} ${last ? styles.last : ''}`;
return (
  <div onClick={onclick} className={indicatorsClassName}>
    <div className={indicatorClassName} key={name}>
      {Icon && <Icon className={`${styles.logo} ${styles.themeLogo}`} />}
      <div className={styles.indicators__name}>{name}</div>
      <div className={styles.indicators__value}>{value}</div>
    </div>
  </div>
);
};

```

Таким же образом были созданы все остальные компоненты веб-приложения. А следующим шагом будет объединение этих компонентов в страницы. Компоненты могут быть переиспользованы на разных страницах или в разных контекстах, что делает архитектуру более гибкой и масштабируемой.

Для объединения компонентов в страницы мы создаем компоненты-контейнеры, которые будут содержать другие компоненты и управлять их взаимодействием и состоянием. Они выглядят как обычные компоненты и могут быть ответственными за получение данных, обработку событий и передачу необходимых пропсов дочерним элементам.

Когда все компоненты и страницы веб-приложения были реализованы, следующим шагом является настройка маршрутизации. Для этого нам понадобится библиотека React Router DOM. Скриншоты реализованных компонентов и страниц представлены в приложении А.

React Router DOM предоставляет набор компонентов, которые помогают определить маршруты в приложении и отображать соответствующие компоненты при переходе по разным URL-адресам.

Перед использованием динамического перехода по страницам без перезагрузки с помощью React Router DOM необходимо настроить маршрутизацию в приложении. В листинге 2 представлен главный компонент App с настроенной маршрутизацией.

Листинг 2 – Главный компонент App

```
import React from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { Route, Routes } from 'react-router-dom';
import { Header } from './components';
import { Account,
Home,
Login,
Rating,
Registration,
Trainer,
PrivacyPolicy}
from './pages';
import { fetchAuthMe, isAuthSelect } from './redux/slices/auth';
export default function App() {
  const dispatch = useDispatch();
  const isAuth = useSelector(isAuthSelect);
  React.useEffect(() => {
    dispatch(fetchAuthMe());
  }, []);
  return (
    <div className='themeContainer wrapper'>
      <Header />
      <Routes>
        <Route path='/' element={<Home />}></Route>
        <Route path='/login' element={<Login />}></Route>
        <Route path='/register' element={<Registration />}></Route>
        <Route path='/trainer' element={<Trainer />}></Route>
        <Route path='/rating' element={<Rating />}></Route>
        <Route path='/account' element={<Account />}></Route>
        <Route path='/privacy-policy' element={<PrivacyPolicy />}></Route>
      </Routes>
    </div>
  );
}
```

Как только маршрутизация настроена, мы можем использовать компонент Link для создания ссылок на разные страницы приложения. Пример использования компонента Link находится в листинге 3.

Листинг 3 – Ссылка на страницу тренажера

```
<Link to='/trainer'>
  <span className={styles.themeText}>
    Тренажер
  </span>
</Link>
```

4.2. Реализация тренировки

После того, как были реализованы все компоненты веб-приложения, настало время создавать логику тренировки ввода текста. Для этого создаем пользовательский хук, который будет служить для управления тренировкой ввода текста.

Хук (hook) – это функция, которая позволяет использовать состояние и другие возможности в функциональных компонентах. Хуки в React предоставляют нам возможность работать с состоянием компонента, обрабатывать эффекты, подписываться на контекст и выполнять другие действия, необходимые для управления компонентами и их поведением.

Один из таких хуков – «useTrainerInput». Данный хук предоставляет набор состояний, функций и эффектов для управления тренировкой в веб-приложении. Он содержит состояния для хранения введенного текста, случайно сгенерированного текста для печати, текущего индекса символа в тексте, времени начала и окончания тренировки, режима строгой проверки ошибок, нажатой клавиши, количества ошибок и других параметров.

С помощью этого хука мы можем управлять всеми процессами тренировки, такими как:

- 1) изменение ввода текста;
- 2) переключение языка текста;
- 3) переключение подсветки клавиатуры и текста;
- 4) переключение режима;
- 5) обработка события нажатия и опуская клавиш, отображая их на визуализированной клавиатуре;
- 6) производить расчеты точности ввода, скорости набора текста и времени тренировки.

В итоге, этот хук является инструментом, который поможет нам добавить функциональность тренировки ввода текста в веб-приложение, сделать

его интерактивным и удобным для пользователей. Исходный код хука «useTrainerInput» находится в листинге 1 приложения Б.

В ходе тренировки слепой печати может быть полезно вести таблицу рейтинга, которая поможет отслеживать прогресс и сравнивать свои результаты с другими пользователями. В этой таблице будет представлена вся статистика о тренировке.

Для создания таблицы рейтинга была использована относительная важность каждого необходимого значения. Распределение весов выглядит следующим образом.

1. Символов в минуту: вес – 0,3. Этот показатель отражает скорость набора текста, однако не считается определяющим для общей оценки тренировки.
2. Символов всего: вес – 0,1. Объем набранного текста имеет наименьший приоритет.
3. Время: вес – 0,1. Затраченное время также учитывается, но его значение не превышает другие аспекты тренировки.
4. Точность: вес – 0,3. Точность печати без ошибок считается существенным фактором при оценке тренировки.
5. Режим: вес – 0,2. Режим тренировки может влиять на результат, но не является наиболее важным показателем.

Общая сумма весов равна 1, что позволяет учесть все аспекты тренировки, но с разной степенью важности. В итоге рейтинг высчитывается по следующей формуле:

$$\begin{aligned} Rating = & (0,3 * charactersPerMinute) + (0,1 * characterCount) \\ & + (0,1 * time) + (0,3 * accuracy) + (0,2 * strictMode), \end{aligned}$$

где «charactersPerMinute» – символов в минуту, «characterCount» – общее количество символов, «time» – затраченное время на тренировку, «accuracy» – точность печати и «strictMode» – включенность строгого режима тренировки.

Анализируя таблицу рейтинга и учитывая эти веса, пользователи смогут более точно оценить свой прогресс и сосредоточить усилия на нужных аспектах, стремясь к желаемым результатам.

4.3. Реализация изменения темы

Темная тема является популярной функциональностью веб-приложений, которая позволяет пользователям изменять внешний вид приложения, делая его более удобным для использования в темных условиях или просто предоставляя альтернативный дизайн. Скриншоты реализации тем представлены в приложении А.

Для реализации изменения темы был создан пользовательский хук «useTheme» (листинг 4), который использует функциональность Redux, а именно Redux Toolkit для сохранения состояния темы и диспетчеризации действий, связанных с ней.

Листинг 4 – Хук «useTheme»

```
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { setTheme } from '../redux/slices/theme.js';
export const useTheme = () => {
  const isDarkTheme = window?.matchMedia(
    '(prefers-color-scheme: dark)'
  ).matches;
  const defaultTheme = isDarkTheme ? 'dark' : 'light';
  const theme = useSelector((state) => state.theme);
  const dispatch = useDispatch();
  useEffect(() => {
    dispatch(
      setTheme(localStorage.getItem(
        'app-theme') || defaultTheme));
  }, [dispatch]);
  useEffect(() => {
    document.documentElement.setAttribute('data-theme', theme);
    localStorage.setItem('app-theme', theme);
  }, [theme]);
  const updateTheme = (newTheme) => {
    dispatch(setTheme(newTheme));
  };
  return { theme, updateTheme };
};
```

Redux – это популярная библиотека для управления состоянием приложения React. Она позволяет централизованно хранить состояние приложения в едином хранилище (store) и обновлять его с помощью действий (actions) и редьюсеров (reducers).

Redux Toolkit – это расширение Redux, которое предоставляет удобные абстракции и инструменты разработки для работы с Redux, такие как упрощенное определение срезов состояния (slices). Исходный код среза состояния темы представлен в листинге 5.

Листинг 5 – Срез состояния темы

```
import { createSlice } from '@reduxjs/toolkit';
const themeSlice = createSlice({
  name: 'theme',
  initialState: localStorage.getItem('app-theme') || 'light',
  reducers: {
    setTheme: (state, action) => {
      return action.payload;
    },
  },
});
export const { setTheme } = themeSlice.actions;
export default themeSlice.reducer;
```

Redux стоит использовать, когда имеются компоненты, которым требуется обмениваться данными. Так как Redux обеспечивает единое и удобное место для хранения и обновления глобального состояния. Также его стоит использовать, когда необходимо обрабатывать сложную логику управления состоянием, например, взаимодействие с сервером.

Использование Redux Toolkit может быть излишним в том случае, когда имеется хук и необходимо использовать его в разных компонентах, и каждый компонент должен иметь свое собственное состояние. Или, когда не нужно передавать состояние или данные между компонентами, то в этом случае использование Redux только усложняет код.

Хук определяет текущую тему, основываясь на предпочтениях пользователя и настройках системы. Затем он использует хук «useSelector» для получения текущего значения темы из хранилища Redux.

С помощью эффектов «useEffect» хук отслеживает изменения значения темы и обновляет стили приложения. Хук также позволяет изменять текущую тему, отправляя соответствующее действие с помощью «dispatch».

4.4. Управление состоянием

Управление состоянием является важной частью разработки веб-приложений, особенно тех, которые требуют функциональности авторизации, регистрации и редактирования данных. Для управления состоянием React-приложения будет использоваться Redux, а именно Redux Toolkit, Axios для выполнения HTTP-запросов к серверу и React Hook Form для управления данными формы.

В самом начале мы определяем Thunk-действия, начальное состояние и срез состояния для управления данными. В срезе определены действия для обновления состояния при выполнении Thunk-действий, а также селектор для проверки авторизации пользователя. Исходный код среза для регистрации приведен в листинге 6.

Листинг 6 – Срез регистрации

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
import axios from '../..//axios';
export const fetchRegister = createAsyncThunk(
  'auth/fetchRegister',
  async (params) => {
    const { data } = await axios.post('/auth/register', params);
    return data;
  }
);
const initialState = {
  data: null,
  status: 'loading',
};
const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    logout: (state) => {
      state.data = null;
    },
  },
  extraReducers: {
    [fetchRegister.pending]: (state) => {
      state.status = 'loading';
      state.data = null;
    },
  },
});
```

```

    [fetchRegister.fulfilled]: (state, action) => {
      state.status = 'loaded';
      state.data = action.payload;
    },
    [fetchRegister.rejected]: (state, action) => {
      state.status = 'error';
      state.data = null;
    },
  },
});
export const isAuthSelect = (state) => Boolean(state.auth.data);
export const authReducer = authSlice.reducer;

```

Далее в компоненте регистрации мы добавляем хуки «useForm» для управления состоянием формы, «useDispatch» и «useSelector» для взаимодействия с Redux-хранилищем. Затем определяем объект с необходимыми данными для формы. После чего определяем функцию, которая будет вызываться при отправке формы, отправлять и проверять данные на валидность (листинг 7).

Листинг 7 – Функция обработки формы

```

const onSubmit = async (values) => {
  const data = await dispatch(fetchRegister(values));
  if (values.password !== values.passwordSecond) {
    reset();
    return setErrorPassVerification('Пароли не совпадают');
  } else if (values.username.length < 2) {
    reset();
    return setErrorName('Укажите корректное имя');
  } else if (values.password.length < 5) {
    reset();
    return setErrorPassword('Пароль должен состоять минимум из 5 символов');
  }

  if (!data.payload) {
    return setErrorMassage('Имя и почта должны быть уникальными');
  }

  if ('token' in data.payload) {
    window.localStorage.setItem('token', data.payload.token);
  }
};

```

Примерно таким же образом были разработаны остальные срезы данных и управления форм.

4.5. Реализация REST API

REST (Representational State Transfer) – это архитектурный стиль взаимодействия компонентов распределенных приложений через сеть. REST представляет собой набор ограничений и рекомендаций, которые учитываются при проектировании распределенных гипермедиа-систем [18].

Основная идея REST API заключается в разделении различных операций (обычно CRUD – создание, чтение, обновление, удаление) при обращении к URL с помощью различных HTTP-запросов:

- 1) GET – используется для получения данных;
- 2) POST – используется для создания новых данных;
- 3) DELETE – используется для удаления данных;
- 4) PUT – используется для полного обновления ресурса;
- 5) PATCH – используется для частичного обновления ресурса.

Все ответы от REST API возвращаются в формате JSON и содержат код состояния.

Для разработки REST API был использован Node.js версии 18.16.0 в сочетании с фреймворком Express.js. Express представляет простую настройку веб-приложений с небольшим количеством зависимостей. Вместе с Node.js также используется пакетный менеджер npm, который предоставляет множество расширений для Node.js.

В самом начале мы должны определить модель, которая описывает схему данных. Эта модель будет использоваться для создания и хранения объектов в базе данных.

Разберем пример создания модели «User». Данная модель описывает схему данных для создания и хранения пользователей в базе данных. В ней находятся поля: имя, оно обязательное и должно быть уникальным, также как и почта, пароль, который должен быть обязательным. А еще имеется

свойство «timestamps», которое по умолчанию указывает на автоматическую генерацию полей для отображения времени создания и обновления записи. В листинге 8 приведен пример модели «User».

Листинг 8 – Модель «User»

```
const UserSchema = new mongoose.Schema(
  {
    username:
      {
        type: String,
        required: true,
        unique: true
      },
    email:
      {
        type: String,
        required: true,
        unique: true
      },
    password:
      {
        type: String,
        required: true
      }
  },
  {Timestamps: true,}
);
```

После того, как мы определили схему данных, то можно приступить к созданию функций обработчиков маршрутов, например, функции «register», для создания пользователей. Функция «register» представляет собой обработчик POST-запроса по пути «/auth/register». Эта функция использует асинхронный подход и ожидает получение данных от клиента через объект «req» и отправляет ответ клиенту через объект «res». В листинге 9 представлена функция «register».

Листинг 9 – Функция «register»

```
export const register = async (req, res) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json(errors.array());
    }
    const passwordNaked = req.body.password;
    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(passwordNaked, salt);
    const doc = new UserModel({
      email: req.body.email,
      username: req.body.username,
```

```

    password: passwordHash,
  });
  const user = await doc.save();
  const token = jwt.sign(
    {
      _id: user._id,
    },
    'secret',
    {
      expiresIn: '30d',
    }
  );
  const { password, ...userData } = user._doc;
  res.json({
    ...userData,
    token,
  });
} catch (error) {
  res.status(500).json({
    message: 'Не удалось зарегистрироваться',
  });
}
};

```

Для регистрации и авторизации пользователей в приложении использовался JSON Web Token – это компактный и самодостаточный способ передачи информации между двумя сторонами в формате JSON. Когда пользователь успешно проходит процесс аутентификации сервер создает JWT токен для пользователя, содержащий определенные данные.

Для хеширования паролей в коде использовался пакет «brypt». Brypt является популярной библиотекой для хеширования паролей в Node.js. Она определяет простой и безопасный способ хранения паролей в базе данных.

Для валидации данных в коде использовался пакет «express-validator». Express validator является популярным пакетом для валидации данных в приложениях Express.js. Использование express-validator упрощает процесс валидации данных, обеспечивая гибкость и возможность определения различных правил валидации для разных сценариев.

Таким же образом была создана модель, описывающая схему данных для тренировок, а также остальные функции обработчиков маршрутов.

Все маршруты для API представлены в таблице 1.

Таблица 1 – API маршруты

Метод	Роутер	Описание
POST	/auth/login	Авторизация пользователей
POST	/auth/register	Регистрация пользователей
GET	/auth/me	Информация о пользователе
GET	/auth/register	Получение всех пользователей
PATCH	/auth/:id	Обновление пользовательских данных
POST	/trainers	Создание тренировки
GET	/trainers	Получение всех тренировок
GET	/trainers/account/	Получение всех тренировок одного пользователя
DELETE	/trainers/:id	Удаление тренировки
DELETE	/trainers	Удаление всех тренировок пользователя

4.6. Подключение к базе данных

Перед тем, как работать с базой данных был использован инструмент Insomnia [19] для проверки и тестирования API-маршрутов (рисунок 18). Данный инструмент позволил убедиться, что запросы и маршруты функционируют должным образом.

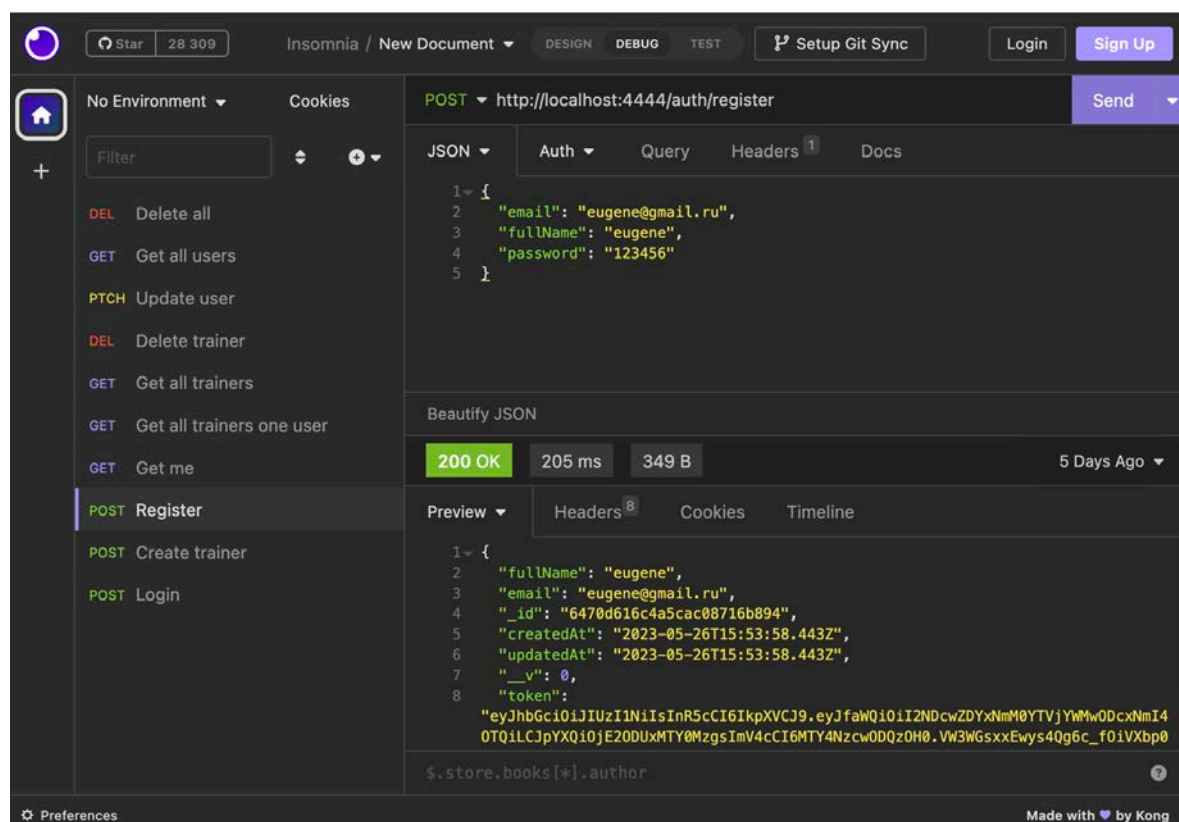


Рисунок 18 – Тестирования API-маршрутов

Для хранения данных была выбрана MongoDB, так как она является документноориентированной базой данных, что не требует фиксированной схемы данных. А для взаимодействия с ней была использована библиотека Mongoose.

Mongoose – это библиотека JavaScript, которая предоставляет удобный способ определения схем и моделей для работы с MongoDB. Подключение к MongoDB представлено в листинге 10.

Листинг 10 – Подключение к базе данных

```
mongoose
  .connect(
    'mongodb+srv://eugene-isaikin:rrfxxb2023@swifttype.gir6c4x.mongodb.net/swift-type?retryWrites=true&w=majority'
  )
  .then(
    () => console.log('database connect')
  )
  .catch(
    (error) => console.log(`database error - ${error}`)
  );
```

Вывод по четвертой главе

В ходе реализации веб-приложения, опираясь на спроектированные макеты, были сверстаны все страницы веб-приложения. Для верстки использовались JSX и SCSS. При помощи React, Redux была реализована возможность взаимодействия пользователя с приложением на клиентской стороне, а при помощи Node.js и Express на стороне сервера.

5. ТЕСТИРОВАНИЕ

После разработки веб-приложения необходимо выполнить тестирование с целью проверки работоспособности и выявления ошибок в работе. Для этого необходимо произвести функциональное и нефункциональное тестирование в соответствии с требованиями.

5.1. Функциональное тестирование

Функциональное тестирование заключается в проверке всех реализованных задач. В тестировании принимало участие десять человек. Результаты функционального тестирования приведены в таблице 2 и совпадают с ожидаемыми результатами.

Таблица 2 – Результаты функционального тестирования

№	Команда	Действие	Ожидаемый результат	Результат теста
1	Открыта главная страница приложения	Пользователь нажимает на кнопку «Начать»	Открывается страница тренажера.	Пройден
2	Открыта страница тренажера	Пользователь тренируется и завершает тренировку печати	Появляются результаты пользователя	Пройден
3	Открыта страница тренажера	Пользователь нажимает на переключатель «Подсветка клавиатуры»	Клавиатура меняет цвет.	Пройден
4	Открыта страница тренажера	Пользователь нажимает на переключатель «Подсветка текста»	Написанный текст меняет цвет и текущий символ становится красным.	Пройден
5	Открыта страница тренажера	Пользователь нажимает на переключатель «Строгий режим»	Включается строгий режим.	Пройден

Окончание таблицы 2

6	Открыта любая страница	Пользователь нажимает на кнопку «Регистрация» и вводит корректные данные	Происходит регистрация нового пользователя и перенаправление на главную страницу	Пройден
7	Открыта любая страница	Пользователь нажимает на кнопку «Авторизация» и вводит корректные данные	Происходит авторизация пользователя и перенаправление на главную страницу	Пройден
8	Открыта любая страница	Пользователь нажимает на иконку «Солнце» или «Луна».	Происходит переключение на другую тему приложения	Пройден
9	Открыта любая страница	Авторизованный пользователь нажимает на кнопку «Выйти»	Происходит выход из учетной записи	Пройден
10	Открыта любая страница	Пользователь нажимает на кнопку «Рейтинг»	Происходит переход на страницу с лучшими тренировками	Пройден
11	Открыта страница «Аккаунт»	Авторизованный пользователь меняет имя, почту или пароль	Происходит изменение данных	Пройден
12	Открыта страница «Аккаунт»	Авторизованный пользователь удаляет статистику по своей тренировке	Происходит удаление данных	Пройден
13	Открыта страница «Аккаунт»	Авторизованный пользователь нажимает на кнопку «Выйти»	Происходит выход из учетной записи и перенаправление на главную страницу	Пройден

5.2. Тестирование кроссбраузерности

Тестирование адаптивности заключается в проверке всех описанных требований нефункциональных задач. Для современного веб-продукта реализация под все популярные браузеры необходимое решение. Тестирование проводилось в следующих браузерах:

- 1) Microsoft Edge, 110.0.1587.50;
- 2) Google Chrome, 114.0.5735.106;
- 3) Mozilla Firefox, 113.0.2;
- 4) Яндекс.Браузер, 23.5.0.2285;
- 5) Safari, 18615.2.9.11.4.

Во всех вышеперечисленных браузерах верстка отображается корректно, никаких ошибок не выявлено. Следовательно, тестирование на отображение в различных браузерах выполнено успешно.

Вывод по пятой главе

Исходя из требований к веб-приложению, определенных в главе 2 было проведено функциональное тестирования приложения со стороны пользователя, а также проведено тестирование кроссбраузерности под ведущие интернет-браузеры.

ЗАКЛЮЧЕНИЕ

В рамках выполнения выпускной квалификационной работы было спроектировано, реализовано и протестировано веб-приложение «Тренажер для набора текста на клавиатуре».

В ходе реализации приложения были решены следующие задачи.

1. Произведен анализ предметной области.
2. Разработаны требования к веб-приложению.
2. Выполнено проектирование веб-приложения.
3. Реализовано веб-приложение.
4. Выполнено тестирование реализованной части.

В результате выполнения выпускной квалификационной работы были решены все поставленные задачи, таким образом, цель данной работы достигнута.

ЛИТЕРАТУРА

1. STAMINA-ONLINE. [Электронный ресурс] URL: <https://stamina-online.com/ru/> (дата обращения: 03.02.2023 г.).
2. Соло на клавиатуре. [Электронный ресурс] URL: <https://solo.nabiraem.ru/> (дата обращения: 04.02.2023 г.).
3. Touch Typing Study. [Электронный ресурс] URL: <https://www.typingstudy.com/> (дата обращения: 05.02.2023 г.).
4. Редактор кода VSCode. [Электронные ресурс] URL: <https://code.visualstudio.com> (дата обращения: 07.02.2023 г.).
5. Веб-сервис для хостинга проектов GitHub. [Электронный ресурс] URL: <https://github.com> (дата обращения: 10.03.2023 г.).
6. Фримен Эрик, Фримен Элизабет. Изучаем HTML, XHTML и CSS. 2-е изд. – СПб.: Питер, 2014 г. – С. 36-40.
7. Дэвид Сойер Макфарланд. Новая большая книга CSS. – СПб.: Питер, 2016 г. – С. 28-47.
8. Препроцессор SCSS. [Электронный ресурс] URL: <https://sass-scss.ru> (дата обращения: 15.03.2023 г.).
9. JavaScript и jQuery. Интерактивная веб-разработка / Джон Дакетт; [пер. с англ. М.А. Райтмана]. – Москва : Издательство «Э», 2018 г. – С. 17-23.
10. React: современные шаблоны для разработки приложений. 2-е изд. – СПб.: Питер, 2022 г. – С. 30-50.
11. React и Redux: функциональная веб-сборка. – СПб.: Питер, 2018 г. – С. 190-200.
12. JavaScript. Полное руководство, 7-е изд.: Пер. с англ. – СПб.: ООО «Диалектика», 2021 г. – С 617-681.
13. Node.js в действии. – СПб.: Питер, 2015 г. – 91-100.

14. MongoDB: полное руководство. Мощная и масштабируемая система управления базами данных / пер. с англ. Д. А. Беликова – М.: ДМК Пресс, 2020 г. – 37-50.
15. Мартин Фаулер. UML. Основы, 3-е издание. – СПб.: Символ-Плюс, 2004 г. – С. 192.
16. Веб-приложение для проектирования интерфейсов Figma. [Электронный ресурс] URL: <https://figma.com/> (дата обращения: 09.02.2023 г.).
17. Design SwiftType. [Электронный ресурс] URL: <https://inlnk.ru/zag1N5> (дата обращения: 10.02.2023 г.).
18. JavaScript и Node.js для веб-разработчиков / Н. А. Прохоренок, В. А. Дронов. – СПб.: БВ-Петербург, 2022 г. – 560-566.
19. API REST клиент Insomnia. [Электронный ресурс] URL: <https://insomnia.rest> (дата обращения: 10.04.2023 г.).

ПРИЛОЖЕНИЯ

Приложение А. Скриншоты веб-приложения



Рисунок 1 – Главная страница (светлая тема)

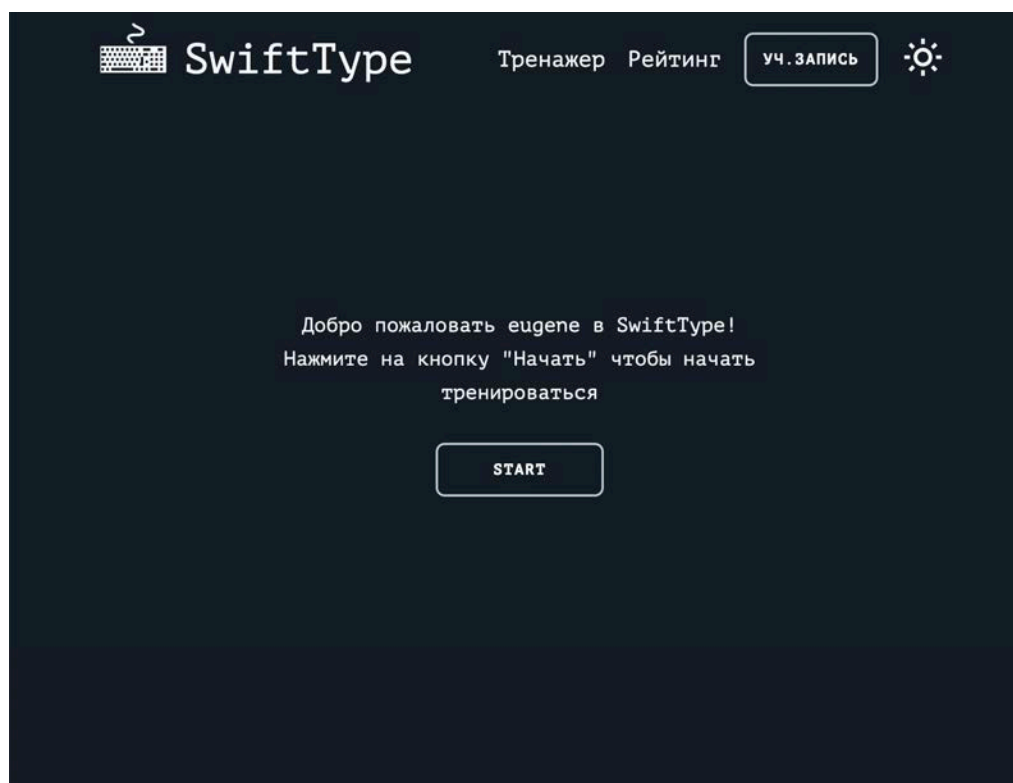


Рисунок 2 – Главная страница (темная тема)

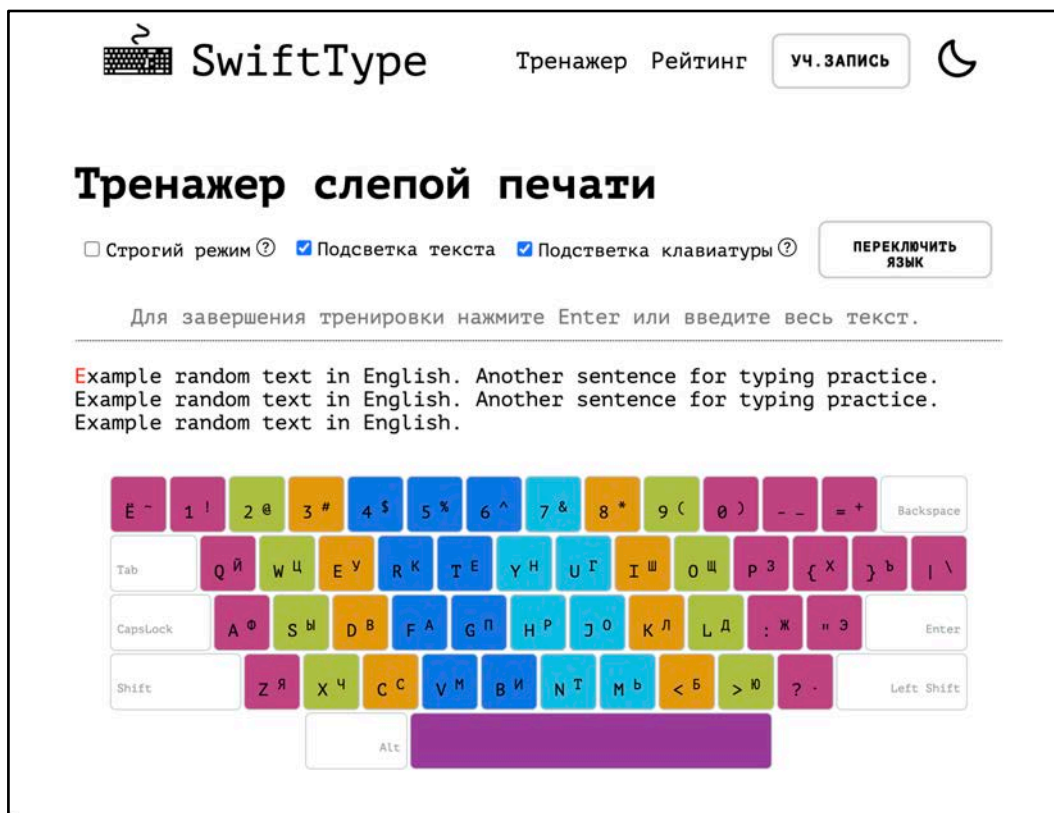


Рисунок 3 – Страница тренажера (светлая тема, заливка текста и клавиш)

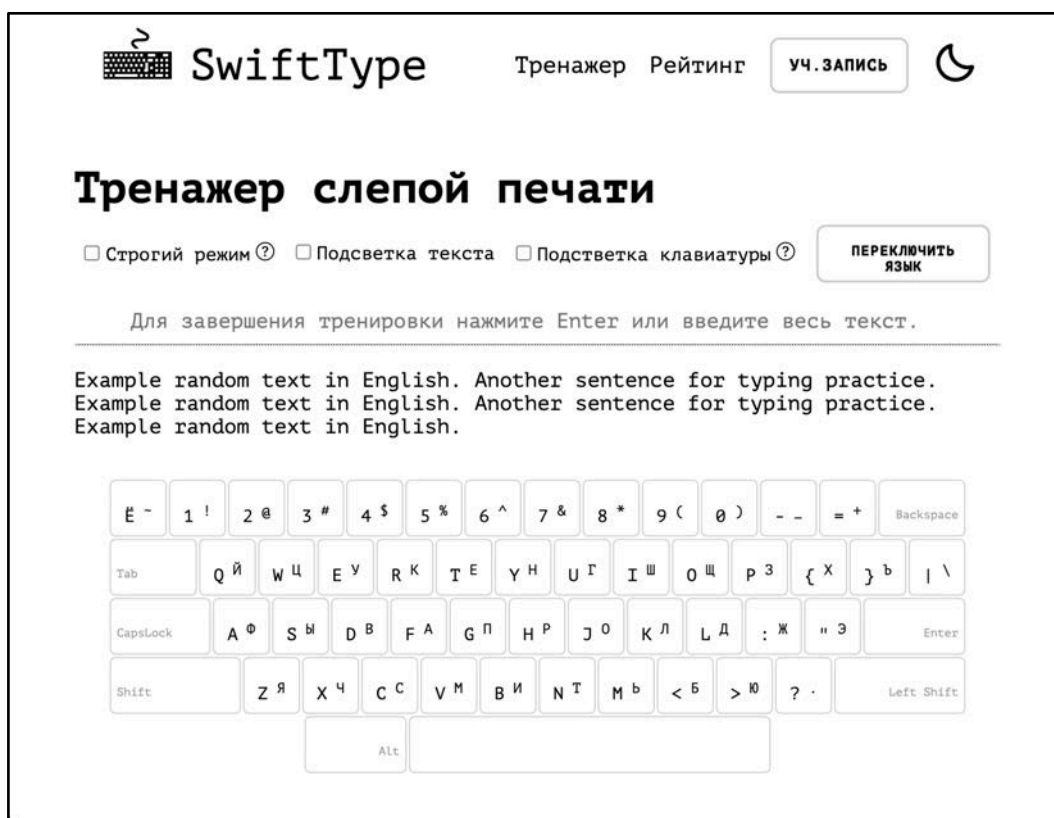


Рисунок 4 – Страница тренажера (светлая тема)

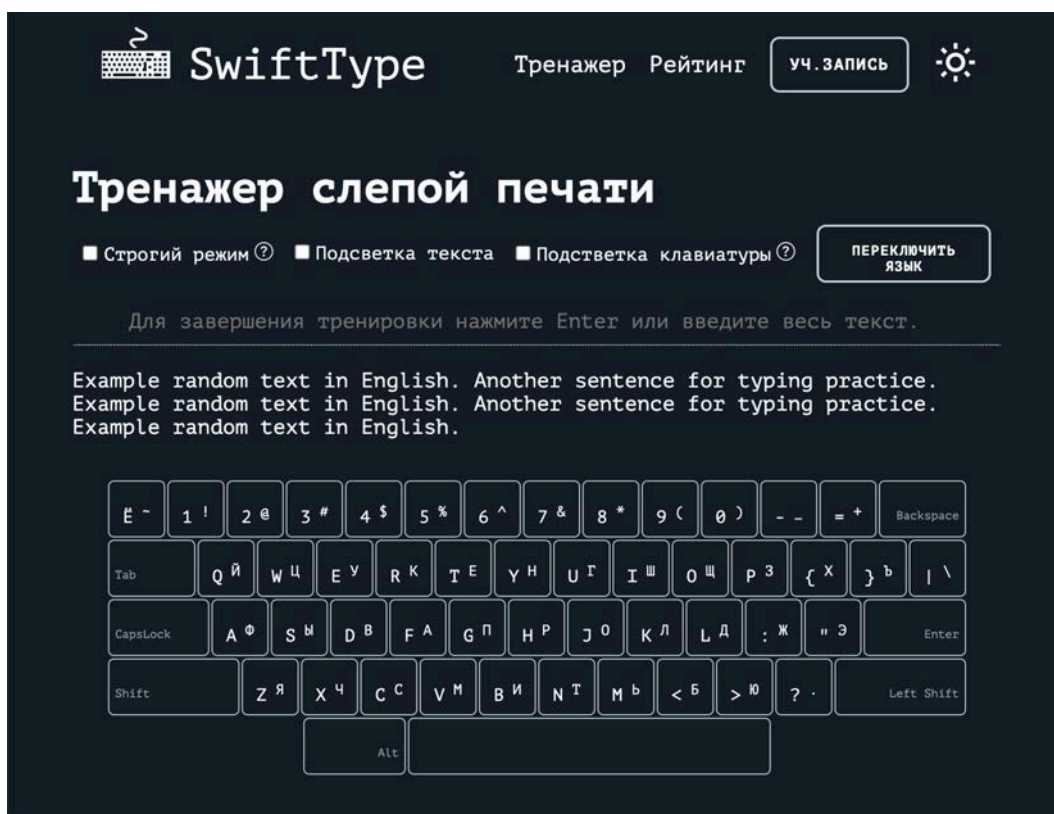


Рисунок 5 – Страница тренажера (темная тема)

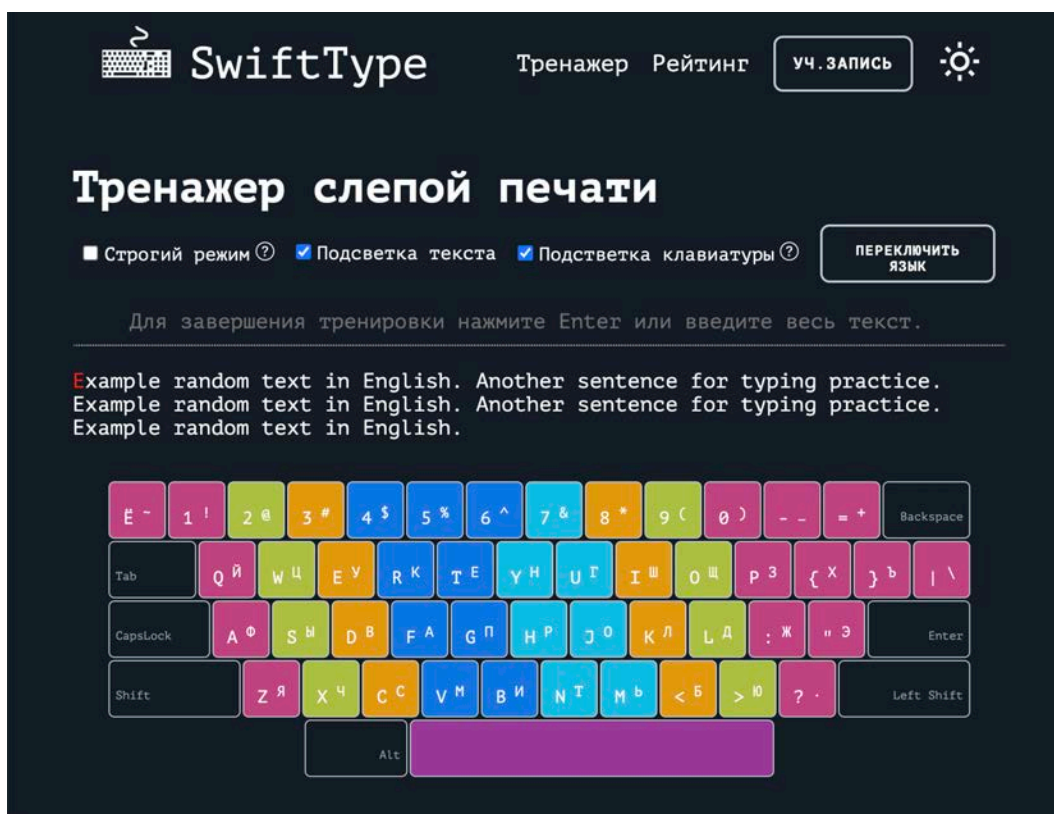


Рисунок 6 – Страница тренажера (темная тема, заливка текста и клавиш)

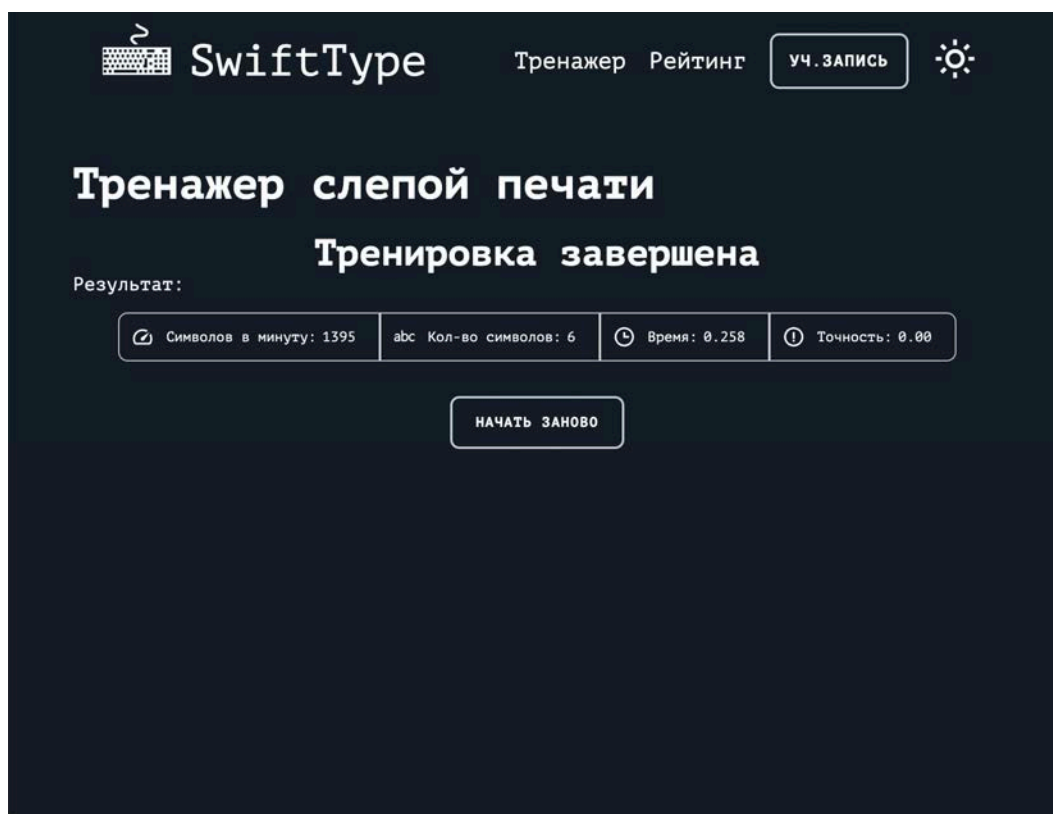


Рисунок 7 – Станица статистики тренировки (темная тема)



Рисунок 8 – Страница статистики (светлая тема)



Рисунок 9 – Страница рейтинга (светлая тема)



Рисунок 10 – Страница рейтинга (темная тема)

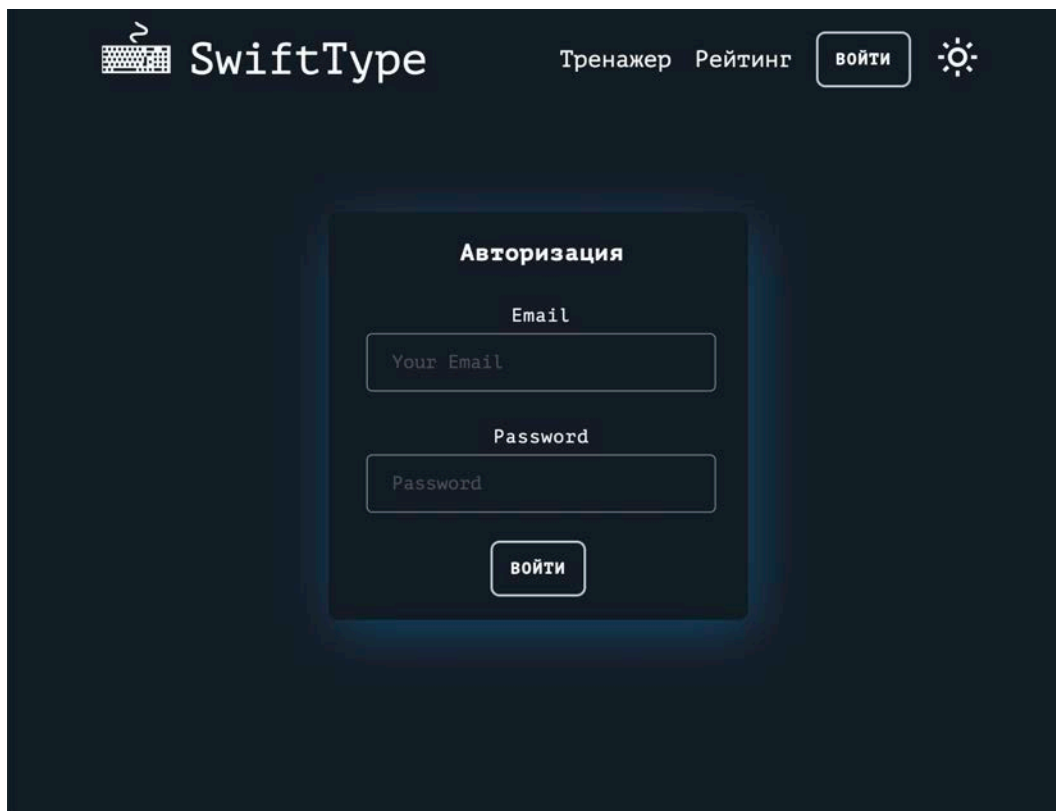


Рисунок 11 – Страница авторизации (темная тема)

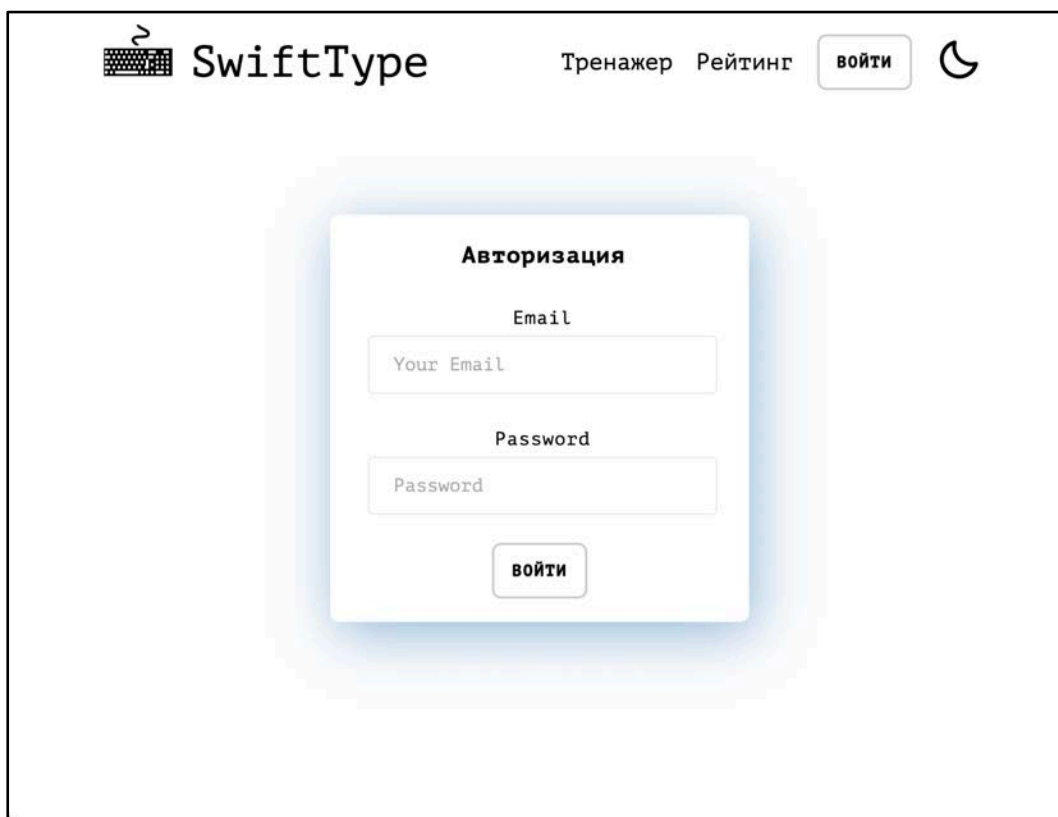
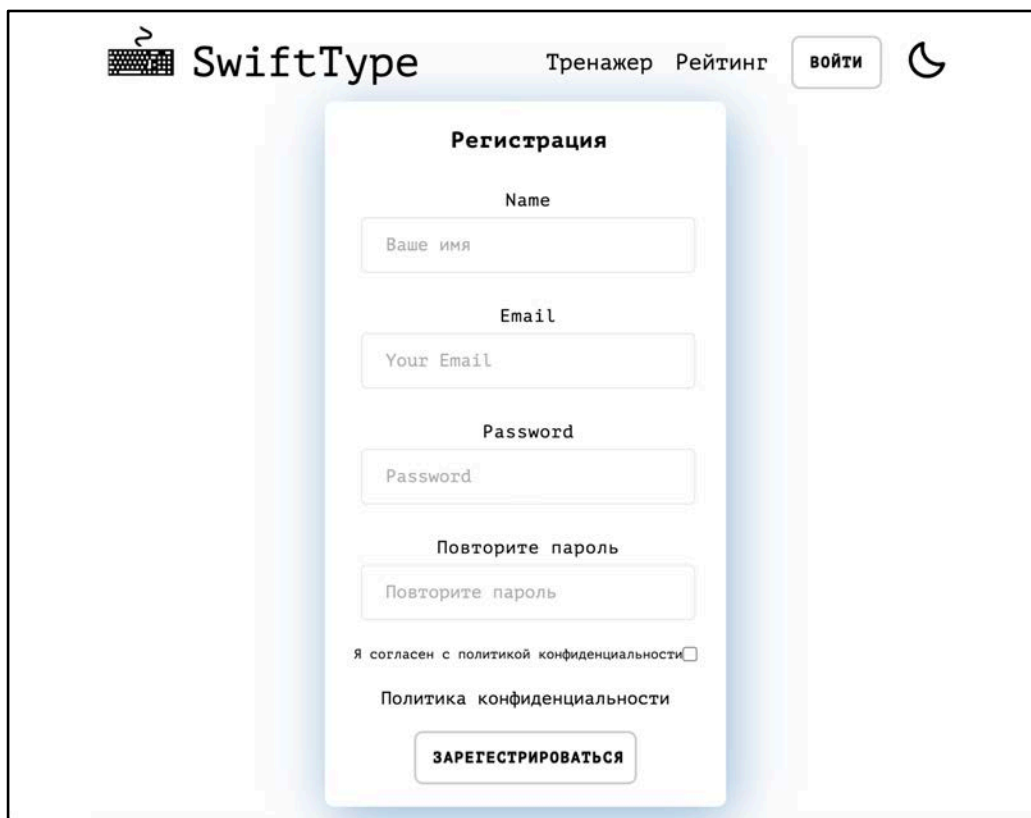
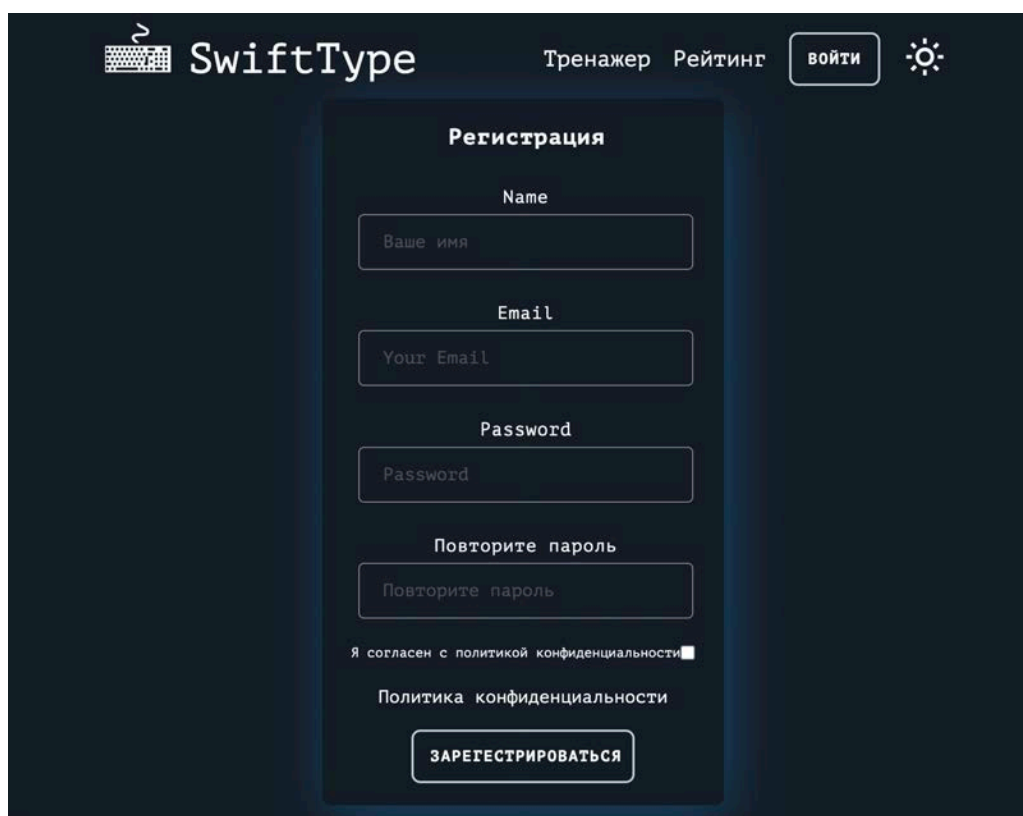


Рисунок 12 – Страница авторизации (светлая тема)



The screenshot shows the registration page of the SwiftType application in a light theme. At the top, there is a header with the SwiftType logo, navigation links for 'Тренажер' (Trainer) and 'Рейтинг' (Rating), a 'войти' (Login) button, and a moon icon for theme toggling. The main content area is a white card with the title 'Регистрация' (Registration). It contains four input fields: 'Name' with placeholder 'Ваше имя', 'Email' with placeholder 'Your Email', 'Password' with placeholder 'Password', and a confirmation field with placeholder 'Повторите пароль'. Below the fields is a checkbox for 'Я согласен с политикой конфиденциальности' (I agree with the privacy policy) and a link to 'Политика конфиденциальности' (Privacy policy). At the bottom of the card is a 'ЗАРЕГИСТРИРОВАТЬСЯ' (Register) button.

Рисунок 13 – Страница регистрации (светлая тема)



The screenshot shows the registration page of the SwiftType application in a dark theme. The layout is identical to the light theme version, but the background and the registration card are dark. The header includes the SwiftType logo, navigation links for 'Тренажер' (Trainer) and 'Рейтинг' (Rating), a 'войти' (Login) button, and a sun icon for theme toggling. The registration card has the title 'Регистрация' (Registration) and contains the same four input fields: 'Name' (placeholder 'Ваше имя'), 'Email' (placeholder 'Your Email'), 'Password' (placeholder 'Password'), and a confirmation field (placeholder 'Повторите пароль'). Below the fields is a checked checkbox for 'Я согласен с политикой конфиденциальности' (I agree with the privacy policy) and a link to 'Политика конфиденциальности' (Privacy policy). At the bottom of the card is a 'ЗАРЕГИСТРИРОВАТЬСЯ' (Register) button.

Рисунок 14 – Страница регистрации (темная тема)

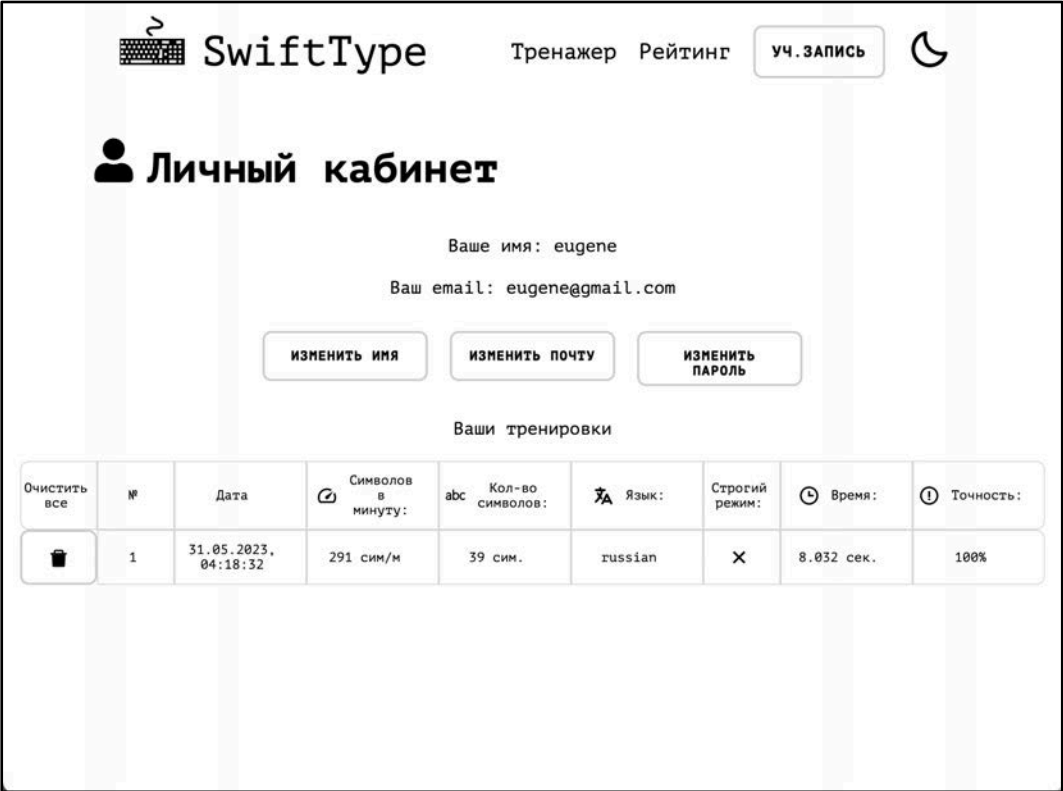


Рисунок 15 – Страница аккаунта пользователя (светлая тема)

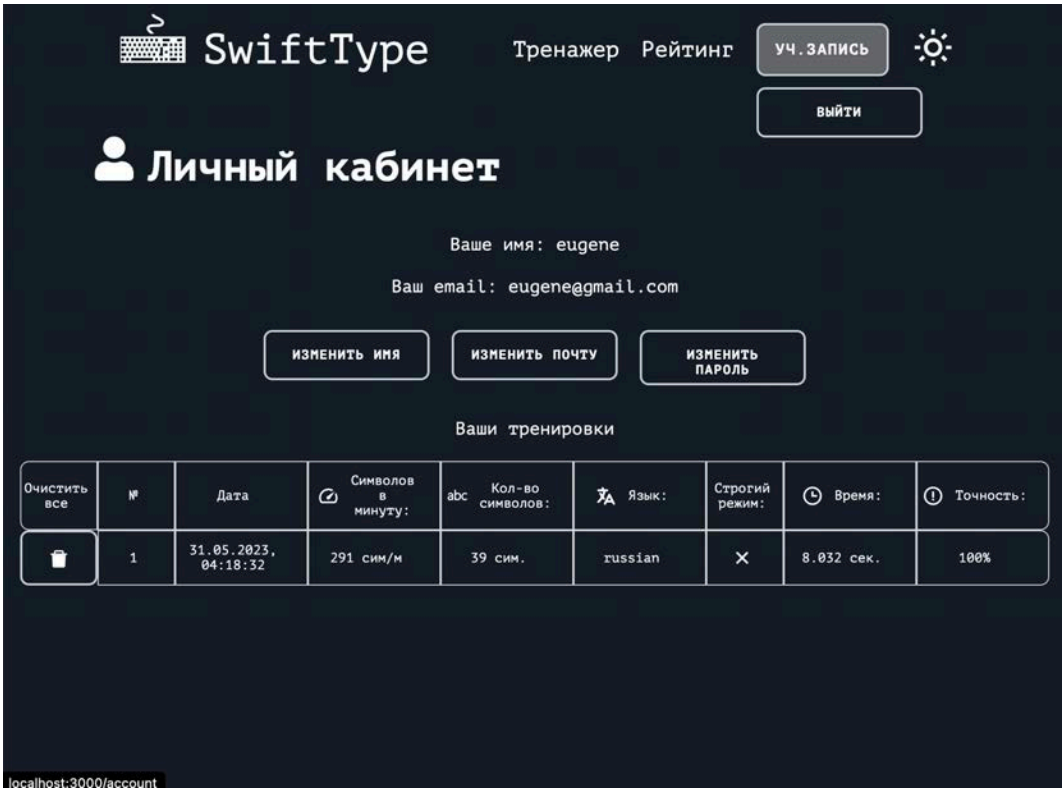


Рисунок 16 – Страница аккаунта пользователя (темная тема)

Приложение Б. Функция для тренировки печати

Листинг 1 – Хук «useTrainerInput»

```
import { useEffect, useRef, useState } from 'react';
/**
 * Хук для управления тренировкой ввода текста.
 * @returns {object} - Возвращает объект с различными функциями и значениями
 для управления тренировкой.
 */
export const useTrainerInput = () => {
  const [inputText, setInputText] = useState(''); // Текущий введенный текст
  const [randomText, setRandomText] = useState(''); // Случайный текст
  const [currentIndex, setCurrentIndex] = useState(0); // Текущий индекс сим-
вола в тексте
  const [startTime, setStartTime] = useState(null); // Время начала трени-
ровки
  const [endTime, setEndTime] = useState(null); // Время окончания тренировки
  const [strictMode, setStrictMode] = useState(false); // Режим строгой про-
верки ошибок
  const [pressedKey, setPressedKey] = useState(null); // Нажатая клавиша
  const [errorCount, setErrorCount] = useState(0); // Количество ошибок
  const [errorCountStrictMode, setErrorCountStrictMode] = useState(0); //
Количество ошибок в строгом режиме
  const inputRef = useRef(null); // Ссылка на поле ввода
  const [language, setLanguage] = useState('russian'); // Язык текста
  const [highlightMode, setHighlightMode] = useState(false); // Режим под-
светки
  useEffect(() => {
    //generateRandomText();
    toggleLanguage();
    inputRef.current.focus();
    document.addEventListener('keydown', handleKeyDown); // Обработчик нажа-
тия клавиши
    document.addEventListener('keyup', handleKeyUp); // Обработчик отпуска-
ния клавиши
    return () => {
      document.removeEventListener('keydown', handleKeyDown); // Удаление
обработчика нажатия клавиши
      document.removeEventListener('keyup', handleKeyUp); // Удаление обра-
ботчика отпускания клавиши
    };
    // eslint-disable-next-line
    // Пустой массив зависимостей означает,
    // что этот эффект должен выполняться только один раз при монтировании
компонента
  }, []);
  useEffect(() => {
    if (!endTime) {
      inputRef.current.focus();
    }
  }, [endTime]);
  //! Переключение языка текста.
  const toggleLanguage = () => {
    setLanguage((prevLanguage) =>
      prevLanguage === 'russian' ? 'english' : 'russian'
    );
    resetTraining();
    generateRandomText();
  };
  //! Генерация случайного текста на основе выбранного языка.
  const generateRandomText = () => {
```

Продолжение листинга 1 приложения Б

```
const russianTexts = [  
  'Мы посетили традиционный гавайский луау, на котором была живая музыка,  
  танцы и вкусная еда.',  
  'Еще одно предложение для тренировки печати.',  
  'Если жить только для себя, своими мелкими заботами о собственном бла-  
  гополучии, то от прожитого не останется и следа.',  
  'Помимо пляжных развлечений, мы также посвятили время исследованию ост-  
  рова.',  
  'Одной из моих любимых частей поездки было знакомство с местной куль-  
  турой.',  
  'Это очень оживленное место, расположенное в Центральной России.',  
  'Однако, есть и некоторые недостатки.',  
  'Прежде всего, он очень грязный и шумный из-за пробок.',  
  'В третьих, жизнь здесь довольно дорогая, и цены домой и квартир до-  
  вольно высокие.',  
];  
  
const englishTexts = [  
  'As there is a student canteen at the university, I often go there for  
  dinner after classes.',  
  'There you may choose between different kinds of soup, meat and fish  
  dishes and desserts.',  
  'I often come home at about seven.',  
  'After supper I do my home work, play computer games and watch TV.',  
  'My mother lays the table.',  
  'It is a family tradition.',  
  'In the morning we gather together in our kitchen at 7 o'clock and have  
  our breakfast.',  
  'He always listens attentively to his patients and gives them his  
  advice.',  
  'We have many relatives.',  
  'My supper is a full meal.',  
];  
const texts = language === 'russian' ? englishTexts : russianTexts;  
const numberOfSentences = 5;  
const randomSentences = [];  
for (let i = 0; i < numberOfSentences; i++) {  
  const randomIndex = Math.floor(Math.random() * texts.length);  
  randomSentences.push(texts[randomIndex]);  
}  
const randomText = randomSentences.join(' ');  
setRandomText(randomText);  
};  
//! Обработчик нажатия клавиши.  
const handleKeyDown = (event) => {  
  if (event.key === randomText[currentIndex]) {  
    handleCorrectChar();  
  } else if (strictMode) {  
    event.preventDefault();  
    setErrorCount((prevCount) => prevCount + 1);  
    if (inputText.length > currentIndex) {  
      setInputText(inputText.slice(0, currentIndex));  
    }  
  }  
  setPressedKey(event.key);  
};  
//! Обработчик отпускания клавиши.  
const handleKeyUp = () => {  
  setPressedKey(null);  
};
```

```

///! Обработчик изменения ввода текста.
const handleInputChange = (event) => {
  const { value } = event.target;
  if (strictMode) {
    const enteredText = value.slice(inputText.length);
    if (enteredText === randomText[currentIndex]) {
      setInputText(value);
      setCurrentIndex((prevIndex) => prevIndex + 1);
      if (value.length === randomText.length) {
        finishTraining();
      }
    } else {
      setErrorCountStrictMode((prevCount) => prevCount + 1);
    }
  } else {
    setInputText(value);
    setCurrentIndex(value.length);
    if (value.length === randomText.length) {
      finishTraining();
    }
  }
  if (value.length === 1 && !startTime) {
    setStartTime(Date.now());
  }
};
///! Обработчик правильного символа.
const handleCorrectChar = () => {
  setCurrentIndex((prevIndex) => prevIndex + 1);
  if (currentIndex === randomText.length - 1) {
    finishTraining();
  }
};
///! Завершение тренировки.
const finishTraining = () => {
  if (startTime) {
    setErrorCount(
      (prevCount) => prevCount + (randomText.length - inputText.length)
    );
    setEndTime(Date.now());
  }
};
///! Сброс тренировки.
const resetTraining = () => {
  setInputText('');
  setCurrentIndex(0);
  setStartTime(null);
  setEndTime(null);
  setErrorCount(0);
  setErrorCountStrictMode(0);
  setTimeout(() => inputRef.current.focus(), 0);
};
///! Расчет точности ввода.
const calculateAccuracy = () => {
  if (strictMode) {
    const accuracy =
      100 -
      (errorCountStrictMode / (inputText.length + errorCountStrictMode)) *
      100;
    return accuracy.toFixed(2);
  }
}

```

```

    let errorCount = 0;
    for (let i = 0; i < inputText.length; i++) {
        if (inputText[i] !== randomText[i]) errorCount++;
    }
    const accuracy =
        ((inputText.length - Math.floor(errorCount)) /
         inputText.length) * 100;
    return accuracy.toFixed(2);
};
//! Расчет скорости набора текста.
const calculateSpeed = () => {
    if (endTime && startTime) {
        const timeInSeconds = (endTime - startTime) / 1000;
        const charactersTyped = inputText.length;
        return Math.floor((charactersTyped / timeInSeconds) * 60);
    }
    return 0;
};
//! Обработчик изменения значения чекбокса "Строгий режим".
const handleCheckboxChange = () => {
    resetTraining();
    setStrictMode(!strictMode);
};
//! Обработчик изменения значения чекбокса "Режим подсветки".
const handleHighlightModeChange = () => {
    setHighlightMode(!highlightMode);
};
//! Обработчик нажатия клавиши Enter.
const handleEnterPress = (event) => {
    if (event.key === 'Enter') {
        event.preventDefault();
        finishTraining();
    }
};
return {
    toggleLanguage,
    inputText,
    randomText,
    language,
    highlightMode,
    currentIndex,
    strictMode,
    pressedKey,
    errorCount,
    errorCountStrictMode,
    inputRef,
    startTime,
    endTime,
    handleKeyDown,
    handleKeyUp,
    handleInputChange,
    handleCheckboxChange,
    handleHighlightModeChange,
    handleEnterPress,
    finishTraining,
    resetTraining,
    calculateAccuracy,
    calculateSpeed,
};
};

```